École Hassania des Travaux Publics

Electrical & Energy Engineering

Real-Time Intelligent Cable Harness Test Bench

FPGA-Based Detection, Edge AI and Cyber-Physical System Integration

Technical Report

Author: Noureddine Bedoui

Program: Electrical & Energy Engineering

Academic Year: 2026–2027

September 20, 2026

Abstract

This project presents an end-to-end intelligent cable harness monitoring and testing system combining FPGA-based real-time signal processing, Edge AI fault classification, and Cyber-Physical System (CPS) technologies.

The proposed architecture separates deterministic signal processing from higher-level intelligent analysis. An FPGA performs real-time signal conditioning and event detection, while a Random Forest model deployed through ONNX Runtime classifies cable conditions using a 20-feature representation. A lightweight Digital Twin maintains the operational state of the monitored system, while MQTT, FastAPI, InfluxDB, and Grafana provide communication, data access, storage, and visualization.

The Edge AI classifier achieved an accuracy of 94.50% and a macro F1-score of 93.80%. The measured mean inference latency was 0.014 197 ms, with a P95 latency of 0.0234 ms. The FPGA implementation achieved positive timing slack with a WNS of 0.093 ns and zero failing timing endpoints.

The complete software architecture was containerized using Docker Compose and validated through an end-to-end execution involving 20 events, including six FPGA-triggered events.

Keywords: FPGA, Edge AI, Digital Signal Processing, Cable Harness, Cyber-Physical Systems, MQTT, Digital Twin, Industrial IoT, ONNX Runtime.

Contents

1 Introduction **1**

1.1 Industrial Context 1
1.2 Problem Statement 2
1.3 Motivation 2
1.4 Objectives 3
1.5 Main Contributions 4
1.5.1 Deterministic FPGA Detection 4
1.5.2 Feature-Based Edge AI 4
1.5.3 Low-Latency Edge Inference 5
1.5.4 CPS-Oriented Integration 5
1.5.5 Industrial IoT Monitoring 5
1.6 Report Organization 5

2 System Architecture **7**

2.1 System Overview 7
2.2 Physical and Acquisition Layer 8
2.3 FPGA Real-Time Processing Layer 8
2.4 Signal Window and Feature Extraction Layer 9
2.5 Edge AI Layer 9
2.6 Cyber-Physical System Layer 10
2.7 Industrial IoT and Monitoring Layer 11
2.8 Data Flow 11
2.9 Software and Deployment Architecture 12
2.10 Architecture Summary 13

3 FPGA-Based Real-Time Detection **14**

3.1 FPGA Design Objectives 14
3.2 FPGA Signal-Processing Architecture 15
3.3 Clocking and FPGA Implementation Architecture 16
3.4 Fixed-Point Representation 17
3.5 Detection Criterion 18

3.6 Sampling and Processing Relationship . . . 19
3.7 Hardware Implementation . . . 19
3.8 Timing Closure . . . 20
3.9 Resource Utilization and Power . . . 21
3.10 Design Verification . . . 22
3.11 FPGA Results Summary . . . 23

4 Signal Feature Extraction **24**
4.1 Feature Extraction Objectives . . . 24
4.2 Signal Window Organization . . . 24
4.3 Feature Categories . . . 25
4.4 Statistical and Amplitude Features . . . 25
4.5 Waveform Variation and Temporal Features . . . 26
4.6 FPGA-Derived Features . . . 28
4.7 Complete Feature Vector . . . 29
4.8 Feature Extraction Implementation . . . 31
4.9 Dataset Organization . . . 31
4.10 Feature Consistency for Edge Inference . . . 32
4.11 Feature Extraction Summary . . . 32

5 Edge AI Fault Classification **34**
5.1 Edge AI Classification Objectives . . . 34
5.2 Edge AI Processing Architecture . . . 35
5.3 Dataset Representation for Classification . . . 35
5.4 Training and Test Strategy . . . 36
5.5 Random Forest Classifier . . . 36
5.6 Classification Performance . . . 37
5.7 Per-Class Classification Results . . . 38
5.8 Confusion Matrix . . . 38
5.9 Feature Importance . . . 39
5.10 Model Export to ONNX . . . 40
5.11 ONNX Runtime Integration . . . 40
5.12 Classification Result and Edge AI Interface . . . 41
5.13 Discussion of Classification Results . . . 42
5.14 Chapter Summary . . . 42

6 Edge Inference Performance **43**
6.1 Inference Performance Objectives . . . 43
6.2 Benchmark Methodology . . . 44
6.3 Benchmark Configuration . . . 44

6.4 Measured Inference Latency . . . 45
6.5 Interpretation of the Measured Latency . . . 45
6.6 Edge Versus Simulated Cloud Reference . . . 46
6.7 Latency and Real-Time Processing Context . . . 46
6.8 Relationship Between Inference and Communication . . . 47
6.9 Benchmark Limitations . . . 47
6.10 Performance Summary . . . 48

7 Cyber-Physical System Integration 49
7.1 CPS Integration Objectives . . . 49
7.2 CPS-Oriented Architecture . . . 50
7.3 Digital Twin Representation . . . 50
7.4 Digital Twin State Model . . . 51
7.5 Digital Twin Update Mechanism . . . 52
7.6 CPS Data Flow . . . 53
7.7 Health Score and Operating Status . . . 53
7.8 Digital Twin Validation . . . 54
7.9 Relationship Between FPGA, Edge AI, and Digital Twin . . . 55
7.10 CPS Integration with the Monitoring Infrastructure . . . 55
7.11 CPS Implementation Scope . . . 56
7.12 Chapter Summary . . . 56

8 Industrial IoT and Monitoring Infrastructure 58
8.1 Chapter Overview . . . 58
8.2 IIoT System Architecture . . . 58
8.3 MQTT-Based Communication Layer . . . 59
8.4 Time-Series Data Storage with InfluxDB . . . 60
8.5 REST Monitoring Interface with FastAPI . . . 61
8.6 Digital Twin and System State Representation . . . 63
8.7 Grafana Monitoring Dashboard . . . 63
8.7.1 Grafana Panel Results . . . 65
8.8 Docker-Based Deployment Infrastructure . . . 67
8.9 End-to-End Monitoring Validation . . . 68
8.10 Chapter Summary . . . 69

9 Dockerized Deployment and End-to-End Validation 71
9.1 Chapter Overview . . . 71
9.2 Docker Compose Deployment . . . 71
9.3 Containerized Service Architecture . . . 72
9.4 Docker Network and Service Ports . . . 72

9.5 Running Container Validation . . . 73
9.6 FastAPI Service Validation . . . 73
9.7 MQTT Communication Validation . . . 74
9.8 InfluxDB Data Storage Validation . . . 74
9.9 Grafana Monitoring Validation . . . 75
9.10 Digital Twin Validation . . . 75
9.11 End-to-End Validation . . . 76
9.12 Deployment Limitations and Observations . . . 76
9.13 Chapter Summary . . . 77

10 Discussion and Limitations **78**
10.1 Chapter Overview . . . 78
10.2 Discussion of the Main Results . . . 78
10.3 System-Level Discussion . . . 79
10.4 Limitations . . . 79
10.5 Chapter Summary . . . 79

11 Future Research Directions **80**
11.1 Chapter Overview . . . 80
11.2 Experimental Dataset Expansion . . . 80
11.3 Advanced Edge AI Models . . . 80
11.4 Enhanced FPGA Processing . . . 81
11.5 Digital Twin Development . . . 81
11.6 Scalable IIoT Deployment . . . 81
11.7 Chapter Summary . . . 81

12 Conclusion **82**
12.1 Chapter Overview . . . 82
12.2 Main Achievements . . . 82
12.3 Overall Contribution . . . 83
12.4 Final Remarks . . . 83

List of Figures

2.1 Overall architecture of the proposed intelligent cable harness test bench. 7

3.1 Simulink implementation of the FPGA real-time signal-processing and detection chain. 15
3.2 Clocking and FPGA implementation architecture of the real-time processing system. 16
3.3 Final FPGA timing summary after timing closure. 20
3.4 FPGA resource utilization report after implementation. 22

5.1 Edge AI training and deployment pipeline. 35
5.2 Confusion matrix of the Random Forest classifier on the test subset. . . 38
5.3 Relative feature importance obtained from the trained Random Forest model. 39

7.1 CPS-oriented integration of the physical processing pipeline, Edge AI, and lightweight Digital Twin. 50
7.2 Main state variables maintained by the lightweight Digital Twin. 52
7.3 Digital Twin update sequence following an Edge AI classification result. 53

8.1 Overall IIoT and monitoring architecture of the cable harness test bench. 59
8.2 MQTT communication architecture and deployed topics. 59
8.3 InfluxDB organization used for cable event time-series storage. 60
8.4 FastAPI REST monitoring interface and implemented endpoints. . . . 62
8.5 Digital Twin state update mechanism. 63
8.6 Grafana monitoring dashboard: Cable Health Score and Detected Events panels. 64
8.7 Grafana monitoring dashboard: current health, FPGA detections, health evolution, and event distribution panels. 64
8.8 Docker Compose deployment architecture and service network. 67
8.9 Docker Compose running services and exposed communication ports. . 68

9.1 Containerized software architecture of the cable harness monitoring system. 72

List of Tables

3.1 Fixed-point representations used in the FPGA processing chain. 18
3.2 FPGA resource utilization after implementation. 20
3.3 FPGA timing results before and after timing closure. 21
3.4 FPGA implementation verification results. 23

4.1 Complete 20-dimensional feature vector used by the Edge AI model. . . 30

5.1 Dataset partition used for Random Forest training and evaluation. . . . 36
5.2 Random Forest classification performance on the test set. 37
5.3 Per-class F1-scores obtained on the test set. 38

6.1 Configuration of the Edge AI inference benchmark. 44
6.2 Measured ONNX Runtime inference latency. 45

7.1 Validation results of the Digital Twin state-update mechanism. 54

8.1 Main information transmitted by the event messages. 60
8.2 InfluxDB storage configuration. 61
8.3 Implemented FastAPI REST endpoints. 61
8.4 Health score evolution during the controlled 20-event run. 65
8.5 Event distribution in the controlled validation sequence. 67
8.6 Main Docker service ports. 67
8.7 Observed Docker Compose service status. 68
8.8 End-to-end monitoring validation results. 69

9.1 Docker services and exposed ports. 72
9.2 Observed Docker Compose service status. 73
9.3 FastAPI endpoints validated during deployment. 73
9.4 End-to-end deployment and monitoring validation results. 76

Chapter 1

Introduction

1.1 Industrial Context

Modern industrial systems increasingly rely on distributed embedded electronics, communication networks, and sensor-driven monitoring architectures. In such systems, electrical cable harnesses are essential for transmitting power and signals between electronic control units, sensors, actuators, and other subsystems. A fault affecting a cable or its electrical connection can therefore propagate to higher-level system functions and lead to incorrect measurements, communication failures, or degraded system availability.

Cable harness testing is consequently an important part of industrial quality control and maintenance. Conventional testing approaches generally rely on electrical continuity measurements, threshold-based checks, or offline analysis. While these methods can detect a number of common faults, they may provide limited information about the nature of the detected anomaly and may not directly support intelligent classification or continuous monitoring.

The development of embedded computing platforms and FPGA-based systems provides an opportunity to perform deterministic signal processing directly at the edge. At the same time, machine learning models can be used to classify signal patterns and distinguish between different types of abnormal behavior. Combining these capabilities with industrial communication and monitoring technologies creates an architecture in which physical measurements, real-time computation, artificial intelligence, and system-level monitoring operate as interconnected components.

This project investigates such an architecture through the development of a real-time intelligent cable harness test bench integrating FPGA-based signal processing, Edge AI inference, and a lightweight Cyber-Physical System (CPS) layer.

1.2 Problem Statement

A cable harness monitoring system must satisfy several requirements simultaneously. First, the acquisition and processing of electrical signals must be sufficiently fast and deterministic to detect short-duration anomalies. Second, the system should provide more information than a simple binary fault indication by identifying different signal conditions. Third, inference should be performed close to the physical system in order to reduce processing and communication overhead. Finally, the resulting information should be accessible through a monitoring infrastructure suitable for industrial applications.

These requirements lead to a multi-stage engineering problem.

At the physical and embedded level, the system must process sampled electrical signals and detect relevant deviations in real time. An FPGA is well suited to this task because its hardware architecture enables deterministic and parallel signal-processing operations.

At the intelligence level, the system must transform signal windows into meaningful numerical features and use these features to classify different operating conditions. The classification problem considered in this project includes normal operation as well as several fault and disturbance classes.

At the system level, the detected events must be transmitted to higher-level software components, associated with a representation of the monitored system state, and exposed through monitoring and application interfaces. This motivates the integration of a lightweight Digital Twin and Industrial IoT communication infrastructure.

The central problem addressed by this work can therefore be formulated as follows:

> How can FPGA-based deterministic signal processing, resource-efficient Edge AI, and CPS-oriented monitoring be integrated into a single architecture for real-time cable harness anomaly detection and classification?

1.3 Motivation

The motivation for the proposed system is based on the complementary capabilities of FPGA processing, machine learning, and cyber-physical integration.

FPGA-based processing provides a deterministic execution environment for operations that must be performed at a fixed sampling rate. In this project, the FPGA is responsible for real-time signal processing and threshold-based event detection. This first stage operates independently of the higher-level software infrastructure and provides information that can subsequently be incorporated into the AI processing pipeline.

Machine learning provides a second level of analysis. Instead of relying exclusively on predefined decision thresholds, a classifier can exploit multiple statistical and temporal

characteristics of a signal window. This makes it possible to distinguish between several classes of signal behavior using a compact feature representation.

Performing inference at the edge is also important for this type of application. Sending raw signal windows to a remote computing system would introduce additional communication and processing overhead. A local inference stage can instead produce a classification result directly from the extracted features. The resulting latency can then be measured independently and compared with a simulated cloud-processing reference used in this project.

Finally, integrating the processing pipeline with a Digital Twin and an Industrial IoT monitoring stack makes it possible to represent the current state of the monitored system and expose information to higher-level applications. This transforms the project from an isolated FPGA or machine-learning experiment into a CPS-oriented engineering architecture.

1.4 Objectives

The main objective of this project is to design and validate a real-time intelligent cable harness test bench combining deterministic FPGA processing, Edge AI, and CPS-oriented monitoring.

The specific objectives are:

1. Develop an FPGA-based signal-processing chain for real-time cable harness anomaly detection.
2. Implement the required fixed-point signal-processing and threshold-based detection operations using a hardware-oriented design flow.
3. Extract a compact set of statistical, temporal, and FPGA-derived features from sampled signal windows.
4. Train and evaluate a machine-learning classifier capable of distinguishing normal operation from several fault and disturbance classes.
5. Convert the trained model to ONNX format and execute inference using ONNX Runtime at the edge.
6. Quantitatively evaluate Edge AI inference latency using repeated measurements.
7. Integrate the inference results with a lightweight Digital Twin representing the state of the monitored cable harness.
8. Implement MQTT-based communication between the edge processing layer and higher-level monitoring services.

9. Provide system-level monitoring through FastAPI, InfluxDB, and Grafana.
10. Package the main components into a Docker-based deployment and validate the complete end-to-end pipeline.

1.5 Main Contributions

The project combines several technologies into a single experimental architecture. The main contributions of the work are summarized below.

1.5.1 Deterministic FPGA Detection

A real-time FPGA processing chain was developed for signal processing and anomaly detection. The implementation uses fixed-point representations and a deterministic processing pipeline designed for a sampling rate of 1 MHz. The FPGA implementation was subsequently evaluated using synthesis, implementation, timing, utilization, power, and design-rule reports.

1.5.2 Feature-Based Edge AI

A feature extraction pipeline was developed to convert each signal window into a 20-dimensional feature vector. The feature set combines statistical signal characteristics, temporal information, and information generated by the FPGA detection stage.

A Random Forest classifier was trained using these features to distinguish five classes:

- normal operation,
- micro-cut,
- noise,
- transient,
- abrupt fault.

The model was evaluated using a scenario-based train/test split, ensuring that samples originating from the same scenario were not directly distributed across both subsets.

1.5.3 Low-Latency Edge Inference

The trained classifier was exported to ONNX and executed using ONNX Runtime. Repeated inference measurements were performed after a warm-up phase in order to characterize execution latency. This provides a quantitative basis for evaluating the computational feasibility of local inference for the proposed application.

1.5.4 CPS-Oriented Integration

The Edge AI results were integrated with a lightweight Digital Twin representing the monitored cable harness. The Digital Twin maintains system-level state information such as health score, operating status, event information, confidence, and FPGA-related detections.

The project therefore implements a CPS-oriented architecture in which observations from the physical-system processing pipeline are transformed into software state information representing the monitored system.

1.5.5 Industrial IoT Monitoring

An MQTT communication layer was implemented to transport event and status information between the edge system and the monitoring infrastructure. FastAPI provides REST interfaces, while InfluxDB and Grafana are used for time-series storage and visualization.

The complete system was subsequently containerized using Docker Compose, allowing the principal components to operate as an integrated software stack.

1.6 Report Organization

The remainder of this report is organized as follows.

Chapter 2 presents the overall system architecture and describes the interaction between the physical acquisition stage, FPGA processing, Edge AI, Digital Twin, and monitoring infrastructure.

Chapter 3 focuses on the FPGA-based real-time detection system, including the signal-processing chain, fixed-point representation, hardware implementation, and implementation results.

Chapter 4 presents the signal feature extraction methodology and describes the 20 features used by the machine-learning pipeline.

Chapter 5 describes the Edge AI classification system, including dataset organization, training methodology, evaluation metrics, and ONNX model generation.

Chapter 6 evaluates Edge AI inference performance through repeated latency measurements and discusses the implications for real-time edge processing.

Chapter 7 presents the CPS-oriented integration and the Digital Twin state-management mechanism.

Chapter 8 describes the Industrial IoT monitoring infrastructure based on MQTT, FastAPI, InfluxDB, and Grafana.

Chapter 9 presents the Dockerized deployment and the end-to-end validation of the complete system.

Chapter 10 discusses the obtained results, limitations, and methodological considerations.

Chapter 11 identifies potential future research directions, including improvements to the experimental dataset, hardware integration, and intelligent fault diagnosis.

Finally, Chapter 12 summarizes the main results and conclusions of the project.

Chapter 2

System Architecture

2.1 System Overview

The proposed system is designed as a multi-stage architecture combining physical signal acquisition, deterministic FPGA processing, feature extraction, Edge AI inference, cyber-physical state representation, and industrial monitoring.

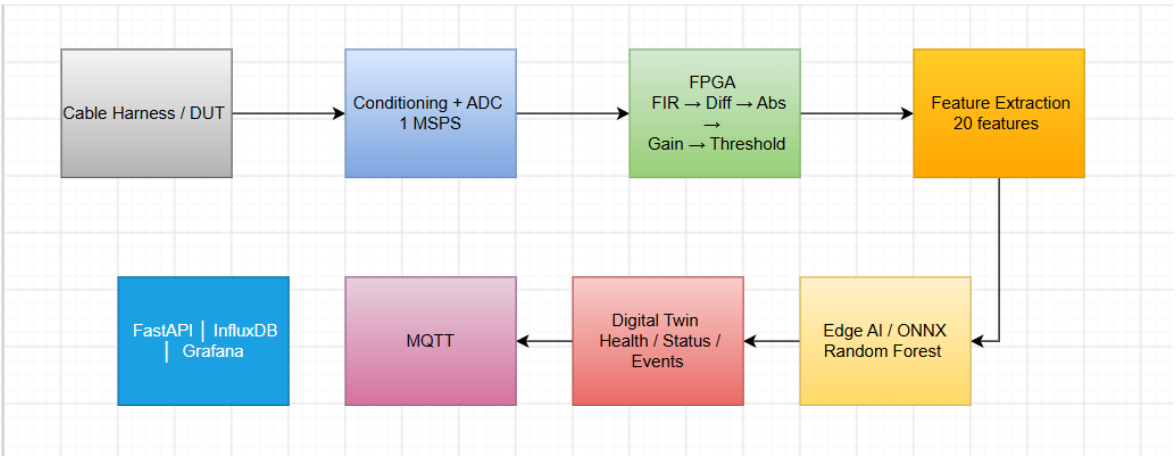


Figure 2.1: Overall architecture of the proposed intelligent cable harness test bench.

Figure 2.1 illustrates the overall architecture and the information flow between the physical acquisition, FPGA, Edge AI, CPS, and monitoring layers.

The overall processing chain follows the sequence:

> Physical Signal → FPGA Detection → Feature Extraction → Edge AI Inference → Digital Twin → Industrial IoT Monitoring

The architecture is organized into four main functional layers:

1. physical and acquisition layer,
2. real-time FPGA processing layer,

3. Edge AI and CPS layer,

4. monitoring and application layer.

This organization separates deterministic signal processing from higher-level software functions while maintaining a continuous information flow between the physical system and the monitoring infrastructure.

2.2 Physical and Acquisition Layer

The physical part of the system represents the cable harness under test and the associated signal acquisition path. The conceptual test-bench architecture consists of a DC supply, cable harness wires, a switching or multiplexing matrix, signal conditioning circuitry, and a multi-channel analog-to-digital conversion stage.

The resulting digital signal is provided to the FPGA processing system. In the current architecture, multiplexing and signal conditioning are used to interface the cable harness with the ADC. Individual shunt measurements on every wire are not required in the implemented configuration.

The acquisition stage operates at a sampling frequency of 1 MHz. Signal windows containing 256 samples are subsequently used by the feature extraction and Edge AI stages.

The physical and acquisition layer therefore provides the interface between the cable harness and the digital processing pipeline.

2.3 FPGA Real-Time Processing Layer

The FPGA constitutes the first computational stage after signal acquisition. Its primary role is to perform deterministic real-time signal processing and detect relevant signal deviations.

The FPGA design uses a system clock of 100 MHz and a processing clock generated at 40 MHz. The signal-processing chain is implemented using fixed-point representations in order to provide predictable hardware behavior and efficient resource utilization.

The main processing sequence is:

FIR Filtering → Fixed-Point Conversion → Unit Delay → Difference → Absolute Value → Gain → Threshold Comparison → Event Detection

The implemented detection criterion is based on the absolute difference between consecutive processed samples. The resulting value is scaled and compared with a predefined threshold. A two-sample validation mechanism is used to avoid declaring an event from a single isolated threshold crossing.

The FPGA provides the final detection output together with information that can be incorporated into the subsequent feature-processing stage. FPGA-derived information used by the higher-level pipeline includes event occurrence and additional event characteristics such as event count, duration, maximum difference, and trigger timing.

The detailed FPGA architecture, numerical representation, timing closure, resource utilization, and implementation results are presented in Chapter 3.

2.4 Signal Window and Feature Extraction Layer

Following signal acquisition and FPGA processing, the sampled data are organized into fixed-size windows of 256 samples.

For each window, a feature extraction process computes a 20-dimensional representation. The feature vector combines three types of information:

- statistical characteristics of the signal,
- temporal and waveform characteristics,
- information generated by the FPGA detection stage.

The resulting feature vector is therefore not limited to conventional statistical descriptors. It also incorporates information from the deterministic embedded processing stage.

The 20 features are subsequently provided to the machine-learning model as a fixed-dimensional input vector. The complete feature definition and extraction methodology are presented in Chapter 4.

2.5 Edge AI Layer

The Edge AI layer performs the classification of the extracted feature vectors.

A Random Forest classifier is used to distinguish five signal conditions:

- normal,
- micro-cut,
- noise,
- transient,
- abrupt.

The trained model is exported to the ONNX format and executed using ONNX Runtime. This provides a standardized inference interface between the trained machine-learning model and the edge application.

The Edge AI stage receives the extracted 20-dimensional feature vector rather than the complete raw signal window. This reduces the amount of information processed by the inference engine and allows the computational cost of inference to be measured independently.

The classification result includes the predicted class and associated confidence information. The result is then passed to the Digital Twin and monitoring infrastructure.

The dataset, training procedure, classification results, and model conversion process are described in Chapter 5, while inference performance is evaluated in Chapter 6.

2.6 Cyber-Physical System Layer

The project incorporates a lightweight Cyber-Physical System architecture by maintaining a software representation of the monitored cable harness.

The Digital Twin receives information from the Edge AI and FPGA processing stages and maintains an evolving representation of the system state. The state includes parameters such as:

- device identifier,
- health score,
- operating status,
- last detected event,
- classification confidence,
- total number of events,
- number of FPGA-triggered events.

Each new classified event updates the state of the Digital Twin. The resulting state represents the current condition inferred from the observations generated by the physical processing pipeline.

The implementation is intentionally lightweight. It should therefore be considered a CPS-oriented architecture with a lightweight Digital Twin rather than a complete industrial digital-twin platform.

The Digital Twin implementation and state-management mechanism are described in Chapter 7.

2.7 Industrial IoT and Monitoring Layer

The upper layer of the architecture provides communication, data storage, visualization, and application interfaces.

MQTT is used as the communication protocol between the edge processing system and the monitoring services. Three principal topics are used:

- `cable/01/events`,
- `cable/01/status`,
- `cable/01/diagnostics`.

Event messages contain information related to the detected condition, classification confidence, timing information, FPGA metadata, and the current Digital Twin state.

Status messages provide the current state of the monitored device, while diagnostic messages provide additional information useful for observing the operation of the edge system.

FastAPI provides REST endpoints for accessing the system state and recorded information. InfluxDB is used for time-series data storage, while Grafana provides visualization of the monitored system.

The monitoring dashboard is organized around six principal views:

1. Cable Health Score,
2. Detected Events,
3. Cable Status / Current Health Score,
4. FPGA Detections,
5. Health Score Evolution,
6. Events by Type.

The complete monitoring infrastructure is described in Chapter 8.

2.8 Data Flow

The complete information flow can be summarized as follows.

First, the cable harness signal is sampled and converted into a digital representation. The FPGA processes the signal in real time and generates deterministic detection information. The sampled signal and FPGA-derived information are then used to construct a feature vector.

The feature vector is provided to the ONNX-based Random Forest classifier running at the edge. The classifier produces a predicted operating condition and confidence information. These results are then passed to the Digital Twin, which updates the current state of the monitored system.

The resulting event and status information is published through MQTT. The FastAPI service receives the information and exposes REST endpoints, while the monitoring infrastructure stores and visualizes the corresponding data.

This flow creates a continuous path from the physical signal to system-level information:

Physical System → Acquisition → FPGA → Features → Edge AI → Digital Twin → MQTT → Monitoring

2.9 Software and Deployment Architecture

The software components are organized as separate functional modules. The Edge AI publisher contains the feature extraction and ONNX inference pipeline, while the Digital Twin provides the system-state representation.

The FastAPI application acts as the software interface to the monitoring system and communicates with the MQTT broker. InfluxDB provides persistent time-series storage and Grafana provides visualization.

The complete stack is containerized using Docker Compose. The main services include:

- MQTT broker,
- FastAPI application,
- Edge AI publisher,
- InfluxDB,
- Grafana.

The services communicate through a dedicated Docker network. Environment variables are used to adapt service addresses between local execution and the containerized deployment.

This organization allows the individual components to be developed and tested separately while providing a reproducible integrated deployment.

The Docker configuration and complete end-to-end validation are presented in Chapter 9.

2.10 Architecture Summary

The resulting architecture combines deterministic embedded processing with software-based intelligence and system monitoring.

The FPGA provides the real-time detection stage, while feature extraction and machine learning provide higher-level signal interpretation. The Digital Twin maintains a software representation of the monitored system, and the MQTT-based monitoring infrastructure provides communication and visualization capabilities.

This layered organization also establishes clear interfaces between the different parts of the system. The FPGA produces detection information, the feature extraction layer produces a fixed-dimensional representation, the Edge AI layer produces classification results, and the CPS/IIoT layers transform these results into persistent and observable system state.

The architecture therefore provides the foundation for the detailed implementation and evaluation presented in the following chapters.

Chapter 3

FPGA-Based Real-Time Detection

3.1 FPGA Design Objectives

The FPGA constitutes the deterministic real-time processing stage of the proposed cable harness test bench. Its primary objective is to detect significant signal variations at the hardware level while providing predictable execution timing and low resource consumption.

The implementation is based on a fixed-point digital signal-processing architecture. Fixed-point arithmetic is used to provide deterministic numerical behavior and to limit the hardware resources required by the processing chain.

The design targets an AMD/Xilinx Artix-7 XC7A100T FPGA in the CSG324 package with speed grade -1. The system is driven by a 100 MHz input system clock, while the signal-processing chain operates at a dedicated 40 MHz clock generated by the `clk_wiz_0` clocking block.

The FPGA processing stage performs the following operations:

- filtering of the acquired signal,
- conversion to the selected fixed-point representation,
- calculation of the sample-to-sample difference,
- extraction of the absolute variation magnitude,
- gain and threshold processing,
- temporal validation of detected events.

The resulting detection information is subsequently combined with the sampled signal and provided to the feature-extraction stage used by the Edge AI pipeline.

3.2 FPGA Signal-Processing Architecture

The implemented FPGA processing chain consists of a sequence of digital signal-processing operations designed to identify significant variations in the acquired cable-harness signal.

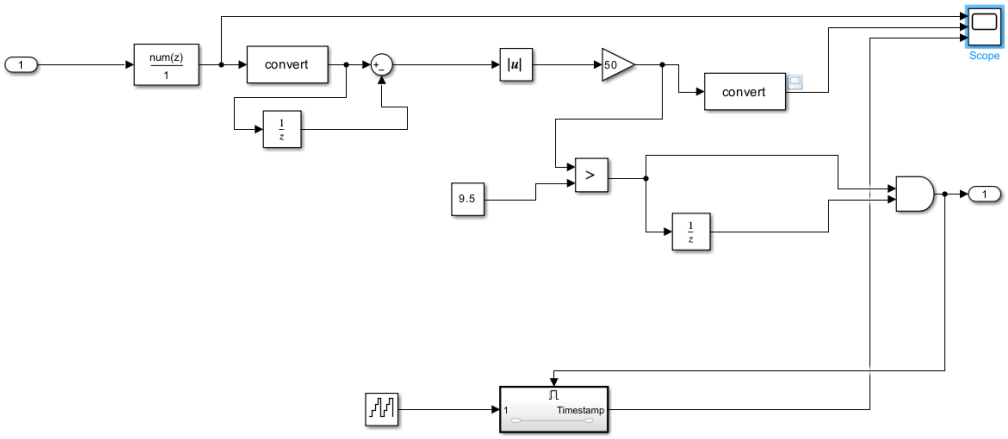


Figure 3.1: Simulink implementation of the FPGA real-time signal-processing and detection chain.

Figure 3.1 presents the functional organization of the implemented processing chain. The main processing sequence is:

FIR Filtering → Fixed-Point Conversion → Unit Delay → Difference → Absolute Value → Gain → Threshold Comparison → Event Detection

The FIR filter first processes the acquired signal. The filtered output is then converted to the fixed-point representation used by the subsequent processing stages.

A unit-delay element stores the previous processed sample. The difference block consequently computes the variation between the current and previous samples. The absolute-value operation converts this signed variation into a magnitude, allowing both positive and negative signal changes to be detected.

The resulting magnitude is multiplied by a gain factor and compared with a predefined threshold. When the threshold condition is satisfied, a candidate detection is generated.

A two-sample validation mechanism is subsequently applied to the detection signal. This temporal validation reduces the sensitivity of the detector to an isolated threshold crossing and provides a simple mechanism for improving event robustness.

The resulting architecture therefore combines filtering, sample-to-sample variation analysis, threshold-based detection, and temporal validation within a deterministic hardware-processing chain.

3.3 Clocking and FPGA Implementation Architecture

The FPGA implementation uses a 100 MHz system clock and a dedicated 40 MHz processing clock. The latter is generated from the input clock using the `clk_wiz_0` clock-management block.

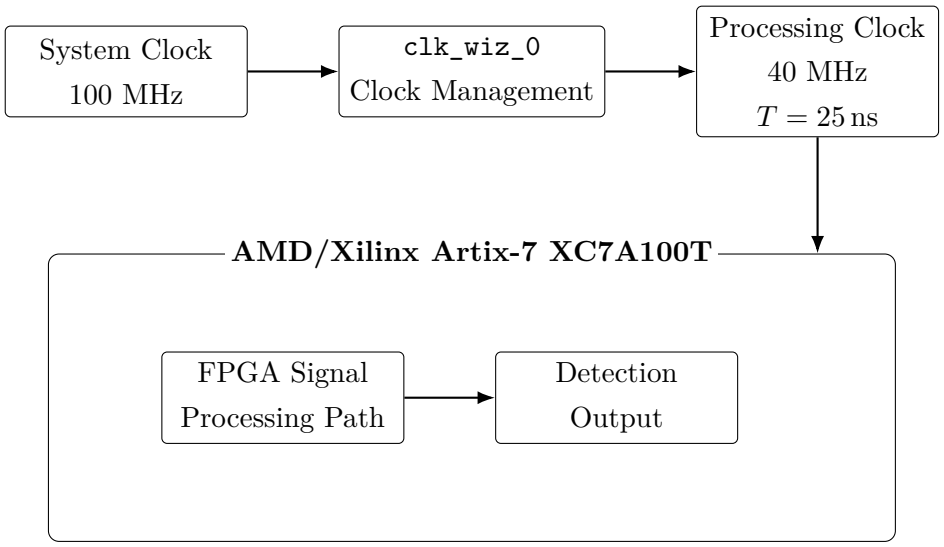


Figure 3.2: Clocking and FPGA implementation architecture of the real-time processing system.

Figure 3.2 summarizes the clocking organization and the relationship between the main elements of the FPGA implementation.

The 100 MHz clock is used as the primary system clock. The `clk_wiz_0` block generates the 40 MHz clock used by the signal-processing chain. The corresponding processing period is:

$$T_{\text{proc}} = \frac{1}{40 \times 10^6} = 25 \text{ ns}.$$

Therefore, each processing-clock cycle has a duration of 25 ns.

Clock-enable and reset signals are used to control the operation of the processing logic. The final FPGA interface exposes the principal input, clock, reset, enable, and detection signals required by the implemented configuration.

The final observable FPGA signals are:

- `In1`,
- `clk`,
- `reset`,

- `clk_enable`,
- `ce_out`,
- `Out3`.

Some signals used during the algorithmic development and reference-model stage are not part of the final FPGA output interface. In particular, the FIR output and scope-related signals are not exposed as final FPGA outputs.

The timestamp was implemented as a 32-bit free-running counter at the reference-model level. However, the timestamp-related logic prevented successful bitstream generation in the final configuration. Consequently, timestamp information remains available at the Simulink/reference-model level rather than as a final FPGA output.

3.4 Fixed-Point Representation

Fixed-point arithmetic is used throughout the principal FPGA processing chain to provide deterministic numerical behavior while controlling the hardware representation of the processed signals.

The input processing representation is defined as:

$$\texttt{fixdt(1,16,12)}.$$

This corresponds to a signed 16-bit representation with 12 fractional bits. The associated scaling factor is:

$$2^{12} = 4096.$$

The gain stage uses a wider representation:

$$\texttt{fixdt(1,32,20)},$$

while the threshold is represented using:

$$\texttt{fixdt(1,24,16)}.$$

The main numerical representations and parameters are summarized in Table 3.1.

Table 3.1: Fixed-point representations used in the FPGA processing chain.

Processing element	Representation	Main parameter
Input conversion	`fixdt(1,16,12)`	Scale = 4096
Gain stage	`fixdt(1,32,20)`	Gain = 50
Threshold	`fixdt(1,24,16)`	Threshold = 9.5

The explicit definition of the numerical formats establishes the signal widths and fractional precision before hardware generation. This contributes to predictable implementation behavior and facilitates the transition from the algorithmic model to the FPGA implementation.

3.5 Detection Criterion

The event-detection mechanism is based on the variation between two consecutive processed samples.

Let $x[n]$ denote the processed signal after FIR filtering and fixed-point conversion. The sample-to-sample difference is defined as:

$$d[n] = x[n] - x[n-1].$$

The magnitude of this variation is then calculated:

$$|d[n]|.$$

A gain of 50 is applied to the resulting magnitude, after which the value is compared with a threshold of 9.5. The detection criterion can therefore be expressed as:

$$50|d[n]| > 9.5.$$

Equivalently, the criterion can be written as:

$$|d[n]| > 0.19.$$

A processed-signal variation greater than this equivalent threshold therefore produces a candidate detection.

The absolute-value operation makes the detection criterion independent of the direction of the variation. Consequently, both positive and negative signal changes can satisfy the threshold condition.

The two-sample validation mechanism is then used to avoid interpreting a single isolated threshold crossing as a complete event. The final detection therefore incorporates

a simple temporal criterion in addition to the amplitude threshold.

3.6 Sampling and Processing Relationship

The signal-acquisition stage operates at a sampling frequency of 1 MHz. The corresponding sampling period is:

$$T_s = \frac{1}{1 \times 10^6} = 1\ \mu\text{s}.$$

Thus, one acquired sample is available every 1 µs, while the FPGA processing clock has a period of 25 ns.

The ratio between the processing-clock frequency and the sampling frequency is:

$$\frac{40\ \text{MHz}}{1\ \text{MHz}} = 40.$$

Consequently, the processing clock provides 40 clock periods during each signal-sampling interval.

This relationship provides a substantially faster internal processing clock than the acquisition rate, allowing the implemented digital processing operations to be executed within the available sampling interval while maintaining a moderate FPGA clock frequency.

3.7 Hardware Implementation

The complete signal-processing architecture was synthesized and implemented for the Artix-7 XC7A100T FPGA.

The implementation uses standard FPGA resources, including lookup tables (LUTs), flip-flops, DSP48 blocks, I/O resources, and clock-management resources.

The final implementation uses:

- 744 LUTs,
- 1039 flip-flops,
- 2 DSP48 blocks,
- 20 I/O resources,
- 1 MMCM.

The detailed utilization results are summarized in Table 3.2.

Table 3.2: FPGA resource utilization after implementation.

Resource	Used	Utilization
LUT	744	1.17%
Flip-flop	1039	0.82%
DSP48	2	0.83%
I/O	20	9.52%
MMCM	1	–

The implemented detection chain therefore occupies only a small fraction of the available logic and arithmetic resources of the selected FPGA. This leaves substantial hardware capacity available for potential extensions of the processing architecture.

3.8 Timing Closure

Timing closure was an important implementation step because the initial FPGA implementation did not satisfy the required timing constraints.

The initial implementation reported:

- WNS $= -13.540\,\text{ns}$,
- TNS $= -34.646\,\text{ns}$,
- 19 failing setup endpoints.

The negative WNS and TNS values indicate that the initial implementation contained setup timing violations.

The design was subsequently modified and re-implemented to improve its timing behavior. The final implementation achieved positive setup and hold margins.

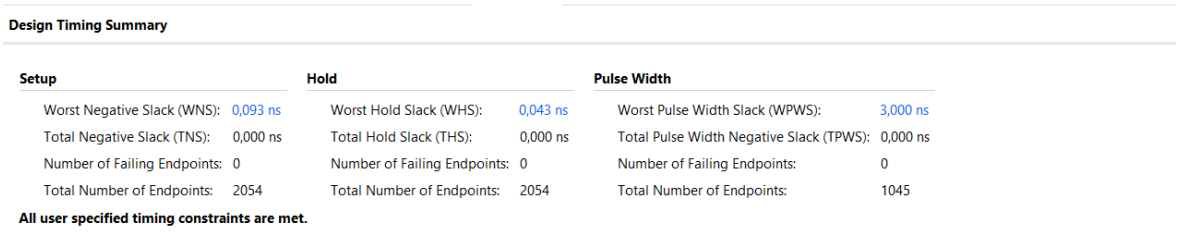


Figure 3.3: Final FPGA timing summary after timing closure.

The timing results before and after the implementation improvements are summarized in Table 3.3.

Table 3.3: FPGA timing results before and after timing closure.

Timing metric	Initial implementation	Final implementation
WNS	−13.540 ns	0.093 ns
TNS	−34.646 ns	0 ns
WHS	–	0.043 ns
Failing endpoints	19	0

The final WNS is 0.093 ns, meaning that the worst setup path satisfies the timing constraint with a positive margin. The final WHS is also positive, at 0.043 ns.

Most importantly, the final implementation reports zero total negative slack and zero failing endpoints. The timing analysis therefore confirms that the final implementation satisfies the reported timing constraints.

The comparison between the initial and final results also demonstrates the importance of the implementation and timing-closure stage in converting the algorithmic design into a timing-clean FPGA implementation.

3.9 Resource Utilization and Power

In addition to timing closure, the final implementation was evaluated in terms of resource consumption and estimated power.

Summary

Resource	Utilization	Available	Utilization %
LUT	744	63400	1.17
FF	1039	126800	0.82
DSP	2	240	0.83
IO	20	210	9.52
MMCM	1	6	16.67

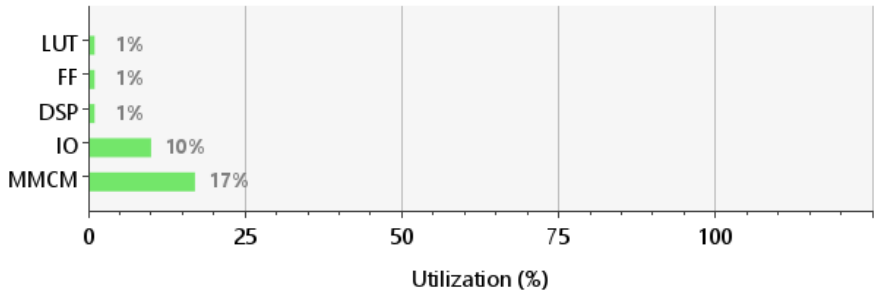


Figure 3.4: FPGA resource utilization report after implementation.

Figure 3.4 provides the implementation resource summary.

The design uses 744 LUTs, corresponding to 1.17% of the available LUT resources, and 1039 flip-flops, corresponding to 0.82%. Only two DSP48 blocks are used, corresponding to 0.83% utilization. The I/O utilization is 9.52%, while one MMCM is used for clock generation.

The relatively low utilization leaves a substantial portion of the FPGA resources available for additional processing functions or future architectural extensions.

The estimated total FPGA power consumption is:

$$P_{\text{total}} = 0.208\,\text{W}.$$

This result indicates that the implemented processing architecture has a relatively low estimated power requirement for the selected FPGA configuration.

3.10 Design Verification

The final FPGA implementation was evaluated using the implementation and design-rule-checking results generated by the development tools.

The final design produced no DRC errors and no critical DRC errors. Six DRC warnings were reported by the implementation tools.

The main verification results are summarized in Table 3.4.

Table 3.4: FPGA implementation verification results.

Verification item	Result
DRC errors	0
Critical DRC errors	0
DRC warnings	6
Timing failing endpoints	0
Total negative slack	0

The absence of DRC errors indicates that no critical design-rule violations were reported for the final implementation. In parallel, the absence of timing failures confirms that the implemented design satisfies the reported timing constraints.

The six DRC warnings remain part of the implementation report and should be reviewed when extending the design or preparing a more production-oriented hardware configuration.

3.11 FPGA Results Summary

The FPGA stage provides the deterministic first-level detection mechanism of the intelligent cable harness test bench.

The final architecture combines FIR filtering, fixed-point processing, sample-to-sample variation analysis, threshold-based detection, and two-sample validation. The acquisition stage operates at 1 MHz, while the internal processing clock operates at 40 MHz.

The implementation initially presented timing violations, with a WNS of -13.540 ns, a TNS of -34.646 ns, and 19 failing setup endpoints. Following the implementation and timing-closure process, the final design achieved a WNS of 0.093 ns, a WHS of 0.043 ns, zero total negative slack, and zero failing endpoints.

The final hardware utilization is also limited, with 744 LUTs, 1039 flip-flops, and two DSP48 blocks used. The estimated total power consumption is 0.208 W.

These results establish the FPGA stage as a lightweight and deterministic processing layer for the proposed system. It provides the first level of real-time event detection while generating information that can be combined with the sampled signal for higher-level feature extraction and classification.

The next chapter focuses on the extraction of the 20-dimensional feature representation used by the Edge AI classification pipeline.

Chapter 4

Signal Feature Extraction

4.1 Feature Extraction Objectives

The feature-extraction stage converts each acquired signal window into a fixed-dimensional numerical representation suitable for Edge AI classification.

The objective is to transform the raw signal and the information generated by the FPGA detection stage into a compact set of descriptors that capture the statistical, waveform, temporal, and event-related characteristics of the cable-harness condition.

Each input sample is represented by a window of 256 signal samples acquired at a sampling frequency of 1 MHz. The resulting feature vector contains 20 features.

The feature-extraction process therefore provides the interface between the deterministic FPGA processing stage and the machine-learning classification stage.

The extracted features are designed to represent three complementary categories:

- statistical and amplitude characteristics of the signal waveform,
- waveform variation and temporal characteristics,
- information generated by the FPGA event detector.

This combination allows the Edge AI model to use both information derived directly from the acquired waveform and information generated by the real-time hardware detector.

4.2 Signal Window Organization

The feature-extraction pipeline operates on fixed-size signal windows.

Each scenario contains 256 consecutive samples, with the signal sampled at 1 MHz. The corresponding observation interval is therefore:

$$T_{\text{window}} = \frac{256}{1 \times 10^6} = 256\ \mu\text{s}.$$

Thus, each feature vector represents a signal observation covering 256 µs.

The use of a fixed window size provides a constant input dimension for the machine-learning model. It also ensures that the same feature-extraction procedure is applied to all signal conditions.

The complete dataset contains 1000 scenarios, with each scenario represented by 256 signal samples. The dataset therefore provides a common input structure for feature extraction, model training, and evaluation.

4.3 Feature Categories

The extracted representation contains 20 features. These features are grouped into three main categories.

The first category describes the statistical and amplitude characteristics of the signal. These features characterize the signal magnitude, amplitude distribution, and overall energy.

The second category describes waveform variation and temporal behavior. These features capture sample-to-sample variations, zero crossings, threshold crossings, event-window duration, deviation from the mean, the temporal position of the signal peak, and changes in slope.

The third category contains information generated by the FPGA detection stage. These features provide a direct connection between the deterministic hardware detector and the machine-learning classifier.

The complete feature vector is:

$$\mathbf{f} = [f_1, f_2, \ldots, f_{20}]. \tag{4.1}$$

The features are maintained in a fixed order so that the same feature representation is used during both model training and ONNX inference.

4.4 Statistical and Amplitude Features

The first group of features describes the statistical and amplitude characteristics of the signal window.

The extracted features in this group include:

- RMS value,
- mean,

- standard deviation,
- peak-to-peak amplitude,
- signal energy,
- crest factor.

The root-mean-square value provides a measure of the effective magnitude of the signal over the observation window. The mean represents its average level, while the standard deviation describes the dispersion of the samples around the mean.

The peak-to-peak amplitude is obtained from the difference between the maximum and minimum sample values:

$$A_{\mathrm{pp}} = x_{\max} - x_{\min}.$$

The signal energy is calculated from the squared samples:

$$E = \sum_{n=0}^{N-1} x[n]^2,$$

where $N = 256$ is the window length.

Finally, the crest factor relates the maximum absolute signal magnitude to its RMS value. It provides information about high-amplitude excursions relative to the overall signal magnitude.

These features provide a compact representation of the amplitude and statistical behavior of the signal without requiring the machine-learning model to process all 256 raw samples directly.

4.5 Waveform Variation and Temporal Features

In addition to statistical and amplitude descriptors, the feature-extraction stage includes features describing waveform variation and temporal behavior.

The corresponding features include:

- maximum slope,
- minimum slope,
- maximum absolute slope,
- zero-crossing count,
- event duration,

- threshold-crossing count,
- maximum deviation,
- time to peak,
- slope change.

For consecutive samples, the discrete sample-to-sample variation is represented by:

$$\Delta x[n] = x[n] - x[n-1].$$

The maximum slope, minimum slope, and maximum absolute slope are obtained from the resulting sequence of sample-to-sample differences. These features characterize the magnitude and direction of rapid signal variations.

The zero-crossing count measures the number of sign changes between consecutive samples within the observation window. It provides an additional description of the waveform behavior.

The `event_duration` feature represents the duration of the complete analysis window in the current implementation. Since each window contains 256 samples acquired at 1 MHz, its value is:

$$T_{\text{event}} = N\Delta t = 256 \times 10^{-6} = 256\ \mu\text{s}.$$

Therefore, this feature currently represents the analysis-window duration rather than the duration of an individually detected abnormal event.

The `threshold_crossings` feature measures the number of sample-to-sample variations whose absolute magnitude exceeds the threshold derived from the FPGA detection parameters. The implemented FPGA parameters are:

$$\text{FPGA_GAIN} = 50$$

and

$$\text{FPGA_THRESHOLD} = 9.5.$$

Consequently, the corresponding signal-domain threshold is:

$$T_{\text{diff}} = \frac{9.5}{50} = 0.19.$$

A threshold crossing is therefore counted when:

$$|x[n] - x[n-1]| > 0.19.$$

The `max_deviation` feature represents the largest absolute deviation of a sample from the mean value of the current signal window. It is calculated as:

$$D_{\max} = \max_n |x[n] - \bar{x}| .$$

The `time_to_peak` feature represents the time position of the maximum absolute signal magnitude within the observation window. The corresponding sample index is obtained from the maximum of $|x[n]|$ and converted into seconds using the sampling interval:

$$t_{\text{peak}} = n_{\text{peak}} \Delta t,$$

where

$$\Delta t = \frac{1}{f_s}.$$

For the implemented sampling frequency of 1 MHz, the sampling interval is 1 µs.

Finally, the `slope_change` feature characterizes the largest absolute change between consecutive sample-to-sample slope values. Since the slope is defined as:

$$\Delta x[n] = x[n] - x[n-1],$$

the slope-change measure is based on:

$$S_{\text{change}} = \max_n |\Delta x[n+1] - \Delta x[n]| .$$

Together, these features provide information about signal variation, waveform structure, temporal position, and rapid changes within the observation window.

4.6 FPGA-Derived Features

A distinctive aspect of the proposed feature representation is the inclusion of information generated by the FPGA detection stage.

Five FPGA-derived features are included:

- `fpga_triggered`,
- `fpga_event_count`,
- `fpga_max_duration`,
- `fpga_max_diff`,
- `fpga_trigger_time`.

The `fpga_triggered` feature indicates whether the FPGA detector generated an event during the observation window.

The `fpga_event_count` feature records the number of FPGA events associated with the observation.

The `fpga_max_duration` feature represents the maximum event duration reported by the FPGA information associated with the signal window.

The `fpga_max_diff` feature represents the maximum sample-to-sample difference reported by the FPGA processing stage.

The `fpga_trigger_time` feature stores the FPGA trigger position as a sample index. It is therefore a sample-domain quantity rather than a Unix timestamp or wall-clock time.

These features establish an explicit interface between the deterministic hardware detection stage and the machine-learning stage. Instead of relying exclusively on waveform-derived descriptors, the classifier also receives information produced by the FPGA detector.

The FPGA-derived features therefore act as embedded descriptors of the first-level hardware interpretation of the signal.

4.7 Complete Feature Vector

The complete 20-dimensional feature vector is maintained in the following order:

Table 4.1: Complete 20-dimensional feature vector used by the Edge AI model.

Index	Feature	Description
1	`rms`	Root-mean-square signal value
2	`mean`	Mean signal value
3	`std`	Standard deviation of the signal
4	`peak_to_peak`	Peak-to-peak signal amplitude
5	`max_slope`	Maximum sample-to-sample slope
6	`min_slope`	Minimum sample-to-sample slope
7	`max_abs_slope`	Maximum absolute sample-to-sample slope
8	`zero_crossings`	Number of sign changes between consecutive samples
9	`signal_energy`	Signal energy over the observation window
10	`crest_factor`	Ratio of maximum absolute magnitude to RMS value
11	`event_duration`	Duration of the complete analysis window
12	`threshold_crossings`	Number of sample-to-sample variations exceeding the FPGA-derived threshold
13	`max_deviation`	Maximum absolute deviation from the window mean
14	`time_to_peak`	Time position of the maximum absolute signal magnitude
15	`slope_change`	Maximum absolute change between consecutive slope values
16	`fpga_triggered`	FPGA event trigger indicator
17	`fpga_event_count`	Number of FPGA events
18	`fpga_max_duration`	Maximum FPGA event duration
19	`fpga_max_diff`	Maximum FPGA sample-to-sample difference
20	`fpga_trigger_time`	FPGA trigger sample index

The resulting vector can therefore be represented compactly as:

$$\mathbf{f} = [f_1, f_2, \ldots, f_{15}, f_{16}, \ldots, f_{20}], \tag{4.2}$$

where the exact feature-to-index mapping is defined in Table 4.1.

This ordering is important because the trained machine-learning model expects the same feature ordering during inference. Changing the order of the features would therefore make the input representation inconsistent with the representation used during model training.

4.8 Feature Extraction Implementation

The feature extraction is implemented as a software processing stage between the signal data and the Edge AI inference engine.

The extraction process receives two principal inputs:

- the 256-sample signal window,
- the FPGA-derived event information associated with the same observation.

The signal-based features are computed from the waveform, while the FPGA-derived features are obtained from the corresponding FPGA information structure.

The two groups are then concatenated into a single 20-dimensional vector.

The resulting processing sequence can be summarized as:

> 256-Sample Signal Window → Signal Feature Computation → FPGA Feature Integration → 20-Dimensional Feature Vector → Edge AI Model

The implementation also performs consistency checks before returning the feature vector. The input signal must contain exactly 256 samples and all samples must be finite. After extraction, the resulting feature vector must contain exactly 20 values, and the extracted values must also be finite.

These checks provide a deterministic interface between the signal-processing stage and the machine-learning inference stage.

4.9 Dataset Organization

The complete dataset contains 1000 signal scenarios. Each scenario consists of 256 samples and belongs to one of five signal-condition classes:

- `normal`,
- `micro_cut`,
- `noise`,
- `transient`,
- `abrupt`.

The five classes represent different operating or fault conditions used to evaluate the ability of the Edge AI stage to distinguish between signal behaviors.

For each scenario, the feature-extraction process generates one 20-dimensional feature vector. The complete feature representation can therefore be considered as a matrix:

$$\mathbf{X} \in \mathbb{R}^{1000\times 20}.$$

The corresponding class labels provide the target output used during supervised machine-learning training and evaluation.

The resulting dataset is subsequently divided into training and test subsets using a scenario-level split. This prevents samples originating from the same scenario from being distributed across both subsets.

The final split contains 800 training scenarios and 200 test scenarios.

4.10 Feature Consistency for Edge Inference

Feature consistency is essential because the machine-learning model is trained using a fixed feature ordering and dimensionality.

Before inference, the extracted vector is therefore checked against the expected model input dimension.

The ONNX model used in the Edge AI stage expects an input named:

$$\texttt{float_input}$$

with a shape of:

$$[\texttt{None}, 20].$$

The feature extractor consequently produces exactly 20 numerical values for each signal window.

The extracted feature matrix is stored using 32-bit floating-point values before being provided to the inference engine. This corresponds to the numerical type expected by the ONNX model input.

This fixed-dimensional interface provides a clear boundary between the feature-extraction software and the ONNX inference engine. It also reduces the risk of inconsistencies between training-time and deployment-time representations.

The same feature ordering used during dataset preparation is retained when the model is executed at the edge.

4.11 Feature Extraction Summary

The feature-extraction stage transforms each 256-sample signal window into a 20-dimensional representation combining statistical and amplitude descriptors, waveform and temporal characteristics, and FPGA-derived information.

The approach avoids passing the complete raw signal directly to the machine-learning model. Instead, the raw waveform is represented using descriptors that capture amplitude, variation, temporal behavior, and information produced by the hardware detection stage.

The resulting representation contains 15 signal-derived features and 5 FPGA-derived features. The 15 signal-derived features describe amplitude, statistical behavior, waveform variation, and temporal characteristics, while the remaining five features provide information generated by the FPGA detection stage.

Across the complete dataset, the process produces 1000 feature vectors corresponding to the 1000 signal scenarios. The resulting 1000×20 representation is then used for supervised training and evaluation of the Random Forest classifier described in the next chapter.

The feature-extraction stage therefore provides a fixed and reproducible interface between the sampled signal, the FPGA detection information, and the Edge AI classification model.

Chapter 5

Edge AI Fault Classification

5.1 Edge AI Classification Objectives

The Edge AI stage is responsible for interpreting the feature representation generated from each signal window and assigning the corresponding operating condition.

The objective is to classify the five signal conditions considered in the dataset:

- `normal`,
- `micro_cut`,
- `noise`,
- `transient`,
- `abrupt`.

Unlike a direct raw-signal classification approach, the proposed system uses the 20-dimensional feature representation described in Chapter 4. This representation combines statistical and waveform characteristics with information produced by the FPGA detection stage.

The classification stage therefore receives a compact and structured representation of the signal rather than the complete 256-sample waveform.

The selected machine-learning model is a Random Forest classifier. The model was selected as a practical tree-based classification method for the extracted feature representation and for its compatibility with deployment through ONNX Runtime.

The complete machine-learning pipeline is organized as:

Signal Windows → Feature Extraction → 20-Dimensional Features →
Scenario-Level Split → Random Forest → ONNX Export → Edge Inference

The following sections describe the organization of this pipeline, the training procedure, the classification results, and the deployment of the resulting model.

5.2 Edge AI Processing Architecture

The complete Edge AI processing architecture is shown in Figure 5.1.

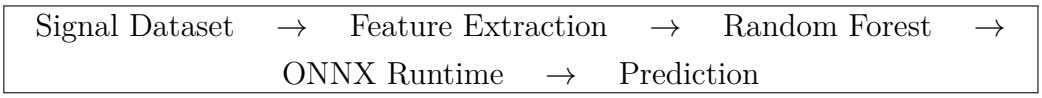


Figure 5.1: Edge AI training and deployment pipeline.

The architecture separates model development from model deployment. During the training stage, the extracted features are organized into a fixed-dimensional dataset and divided into training and test subsets. The Random Forest classifier is trained using the training scenarios and evaluated using the independent test scenarios. After training, the classifier is exported to the ONNX format. The resulting model is then loaded by ONNX Runtime in the edge application.

During deployment, a new signal window is transformed into the same 20-dimensional feature representation used during training. The resulting feature vector is passed to the ONNX model, which produces a predicted class and associated probability information. This separation provides a clear interface between the machine-learning development environment and the edge inference application.

The architecture separates model development from model deployment.

During the training stage, the extracted features are organized into a fixed-dimensional dataset and divided into training and test subsets. The Random Forest classifier is trained using the training scenarios and evaluated using the independent test scenarios.

After training, the classifier is exported to the ONNX format. The resulting model is then loaded by ONNX Runtime in the edge application.

During deployment, a new signal window is transformed into the same 20-dimensional feature representation used during training. The resulting feature vector is passed to the ONNX model, which produces a predicted class and associated probability information.

This separation provides a clear interface between the machine-learning development environment and the edge inference application.

5.3 Dataset Representation for Classification

The complete dataset contains 1000 signal scenarios. Each scenario contains 256 samples and belongs to one of the five considered classes.

After feature extraction, each scenario is represented by a 20-dimensional feature vector. The resulting feature matrix can therefore be represented as:

$$\mathbf{X} \in \mathbb{R}^{1000\times 20}.$$

The corresponding target vector contains one class label for each scenario:

$$\mathbf{y} = [y_1, y_2, \ldots, y_{1000}].$$

The classification problem can consequently be formulated as the estimation of a mapping:

$$f : \mathbb{R}^{20} \rightarrow \{\texttt{normal}, \texttt{micro_cut}, \texttt{noise}, \texttt{transient}, \texttt{abrupt}\}.$$

The use of the extracted representation reduces the dimensionality of each observation from 256 raw samples to 20 descriptive variables.

The feature ordering remains fixed throughout the complete pipeline. This is essential because the Random Forest model is trained using a specific feature ordering and the same ordering must be reproduced during edge inference.

5.4 Training and Test Strategy

The dataset is divided into training and test subsets using a scenario-level split.

A total of 800 scenarios are used for training, while 200 scenarios are reserved for testing.

The split is performed using a group-based strategy. The scenario identity is used as the grouping variable so that samples originating from the same scenario are not distributed between the training and test subsets.

This organization reduces the risk of evaluating the model on information directly associated with a scenario that has already contributed to the training data.

The resulting partition can be summarized as:

Table 5.1: Dataset partition used for Random Forest training and evaluation.

Subset	Number of scenarios	Role
Training	800	Model training
Test	200	Independent evaluation
Total	1000	Complete dataset

The test subset is therefore kept separate from the training process and is used to measure the classification performance reported in this chapter.

5.5 Random Forest Classifier

The selected classifier is a Random Forest model.

Random Forest is an ensemble method based on multiple decision trees. Each tree provides a classification decision and the ensemble combines the individual tree outputs to obtain the final prediction.

For the present application, the model receives the 20-dimensional feature vector as input and predicts one of the five signal-condition classes.

The use of a tree-based model is also convenient for the heterogeneous feature representation used in this project. The feature vector contains statistical values, waveform descriptors, temporal quantities, and FPGA-derived indicators. These variables can therefore be processed directly by the classifier without requiring the raw waveform to be supplied to the model.

The trained classifier is subsequently converted to the ONNX format for deployment.

The machine-learning stage can therefore be separated into two distinct operations:

1. supervised model training using the extracted feature dataset,
2. deployment of the trained model for edge inference.

This separation also allows the computational cost of model training to remain independent from the runtime inference process performed at the edge.

5.6 Classification Performance

The trained Random Forest classifier was evaluated on the 200-scenario test subset.

The resulting overall performance is summarized in Table 5.2.

Table 5.2: Random Forest classification performance on the test set.

Metric	Value
Accuracy	0.9450
Macro Precision	0.9421
Macro Recall	0.9405
Macro F1-score	0.9380

The measured accuracy is therefore 94.50% on the independent test subset.

The macro-averaged precision, recall, and F1-score are also close to the overall accuracy. This indicates that the reported performance is not based only on the aggregate number of correctly classified samples, but is also reflected in the class-averaged performance metrics.

The macro F1-score of 0.9380 provides a compact measure of the balance between precision and recall across the five classes.

5.7 Per-Class Classification Results

The classification performance differs between the five signal conditions.

The measured F1-scores are:

Table 5.3: Per-class F1-scores obtained on the test set.

Class	F1-score
`normal`	0.849
`micro_cut`	1.000
`noise`	1.000
`transient`	1.000
`abrupt`	1.000

The four abnormal-condition classes achieve an F1-score of 1.000 on the evaluated test subset, while the `normal` class obtains an F1-score of 0.849.

This difference is important when interpreting the overall classification results. The aggregate accuracy alone does not reveal the distribution of classification errors between the classes.

The lower F1-score observed for the `normal` class indicates that the remaining classification errors are associated with the distinction between normal observations and one or more of the abnormal classes.

The detailed distribution of these predictions is shown by the confusion matrix in Figure 5.2.

5.8 Confusion Matrix

The confusion matrix provides a class-by-class representation of the predictions obtained on the test subset.

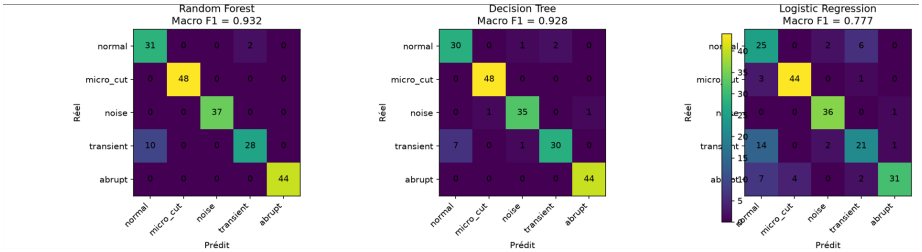


Figure 5.2: Confusion matrix of the Random Forest classifier on the test subset.

The confusion matrix should be interpreted together with the numerical metrics presented previously.

In particular, the matrix makes it possible to identify which signal conditions are confused with the `normal` class and whether any confusion occurs between the abnormal classes.

Because the four abnormal classes obtain an F1-score of 1.000 in the current evaluation, the principal classification limitation visible in the aggregate metrics is associated with the `normal` class.

The confusion matrix therefore provides a more detailed view of the classification behavior than accuracy alone.

5.9 Feature Importance

The Random Forest model also provides a measure of feature importance that can be used to examine the relative contribution of the input variables to the learned classification model.

The feature-importance analysis is useful in the context of this project because the input representation combines conventional signal descriptors with FPGA-derived information.

The resulting feature-importance plot is shown in Figure 5.3.

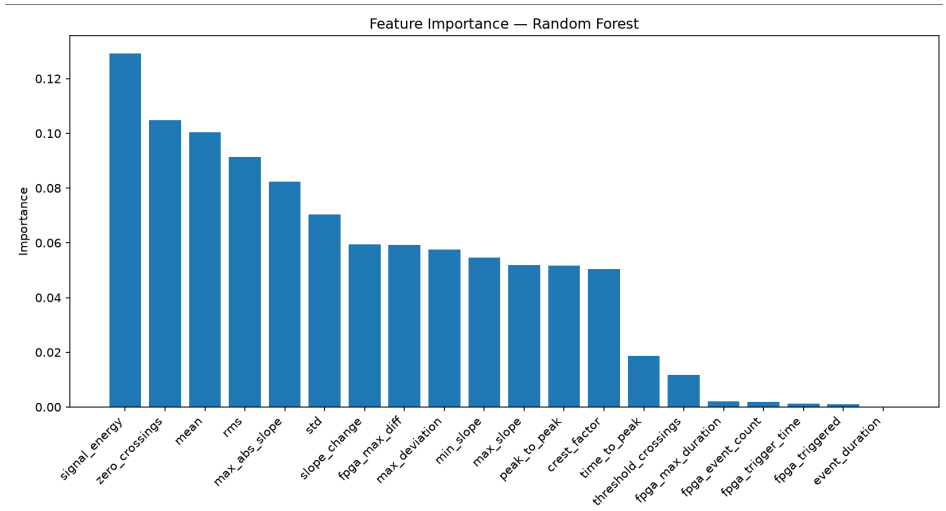


Figure 5.3: Relative feature importance obtained from the trained Random Forest model.

The feature-importance analysis provides an additional interpretation of the trained model by indicating which variables contribute most strongly to the ensemble's decision process.

In particular, the analysis can be used to examine the contribution of the FPGA-derived features relative to the waveform-based descriptors. This is relevant to the proposed architecture because the feature vector intentionally combines software-computed signal characteristics with information generated by the deterministic hardware detector.

The importance values should, however, be interpreted as model-specific measures rather than as direct physical causality. A high feature importance indicates that the feature contributes strongly to the Random Forest decision process, but it does not by itself establish that the corresponding physical quantity is the fundamental cause of a particular fault condition.

5.10 Model Export to ONNX

After training and evaluation, the Random Forest model is exported to the ONNX format.

The exported model provides a standardized representation that can be executed independently from the original training environment.

The ONNX model used in the project exposes the input:

`float_input`

with the expected shape:

[`None`, 20].

The first dimension is variable because the inference engine can accept a batch of observations, while the second dimension is fixed to the 20 features defined by the feature-extraction stage.

The model input uses the floating-point tensor type expected by the ONNX inference interface.

The model produces two outputs:

- the predicted class label,
- probability information associated with the prediction.

The corresponding ONNX outputs are identified as `output_label` and `output_probability`.

This interface provides a direct connection between the feature-extraction module and the edge inference application.

5.11 ONNX Runtime Integration

The exported model is executed using ONNX Runtime.

The edge application first loads the ONNX model and verifies the expected input dimension. The feature-extraction stage then generates the 20-dimensional input vector for each observation.

The resulting vector is converted to the floating-point representation required by the model and supplied to the ONNX Runtime inference session.

The inference sequence can be summarized as:

> Signal Window → 20 Features → Float Tensor → ONNX Runtime → Class Label + Probability

This deployment architecture allows the trained classifier to be executed independently of the model-development environment.

It also provides a clean interface between the machine-learning stage and the subsequent CPS/IIoT components. The predicted class and confidence information can be forwarded to the Digital Twin and published through the MQTT communication layer.

The computational performance of this inference stage is evaluated separately in Chapter 6.

5.12 Classification Result and Edge AI Interface

The Edge AI stage produces two principal pieces of information for each classified observation:

- the predicted signal condition,
- the associated confidence information.

These outputs form the interface between the machine-learning model and the CPS-oriented part of the system.

The predicted condition is used by the Digital Twin to update the current state of the monitored cable harness. The confidence value is also propagated to the higher-level monitoring infrastructure.

The Edge AI output therefore does not remain isolated within the machine-learning module. Instead, it becomes part of the system-level information flow:

> Random Forest → ONNX Runtime → Classification Result → Digital Twin → MQTT → Monitoring

This integration is important because the objective of the project is not limited to classification performance. The classifier is intended to operate as one computational stage within a larger real-time and cyber-physical monitoring architecture.

5.13 Discussion of Classification Results

The Random Forest classifier achieves an accuracy of 0.9450 and a macro F1-score of 0.9380 on the test subset.

The per-class results show a clear difference between the `normal` class and the four abnormal classes. The abnormal classes obtain an F1-score of 1.000 in the evaluated test set, while the `normal` class obtains an F1-score of 0.849.

This behavior suggests that the extracted feature representation provides strong separation for the abnormal signal conditions represented in the dataset. At the same time, the lower performance of the `normal` class indicates that some observations are more difficult to distinguish from abnormal conditions.

The results should be interpreted within the scope of the current dataset and experimental configuration. In particular, the reported values represent performance on the available 1000-scenario dataset and should not be interpreted as a direct estimate of performance under all possible real-world cable-harness conditions.

The classification results nevertheless demonstrate that the proposed feature-based Edge AI stage can provide a practical higher-level interpretation of the deterministic FPGA detection pipeline.

5.14 Chapter Summary

This chapter presented the Edge AI classification stage of the proposed cable-harness test bench.

The 1000 signal scenarios were transformed into a fixed 1000×20 feature representation using the feature-extraction procedure described in Chapter 4. A scenario-level split was used to obtain 800 training scenarios and 200 independent test scenarios.

A Random Forest classifier was trained using the extracted features and evaluated on the test subset. The resulting accuracy was 0.9450, with macro precision of 0.9421, macro recall of 0.9405, and macro F1-score of 0.9380.

The per-class results showed F1-scores of 1.000 for `micro_cut`, `noise`, `transient`, and `abrupt`, while the `normal` class obtained an F1-score of 0.849.

The trained model was exported to ONNX and integrated with ONNX Runtime. The resulting interface expects a 20-dimensional floating-point feature vector and produces a predicted class together with probability information.

The Edge AI output subsequently forms the input to the Digital Twin and IIoT monitoring stages. The computational latency of this inference process is evaluated separately in Chapter 6.

Chapter 6

Edge Inference Performance

6.1 Inference Performance Objectives

The objective of this chapter is to evaluate the computational performance of the Edge AI inference stage after deployment of the trained Random Forest model in ONNX format.

The evaluation focuses specifically on inference latency. Model training time, dataset preparation time, and communication latency are outside the scope of this measurement.

The purpose of the benchmark is to characterize the computational cost of performing classification locally and to determine whether the deployed model introduces a relatively small processing overhead within the proposed edge-processing architecture.

The inference benchmark was performed using ONNX Runtime and was separated into a warm-up phase and a measured phase. This organization reduces the influence of initial runtime effects on the reported measurements.

The main performance indicators considered are:

- mean inference latency,
- median inference latency,
- latency standard deviation,
- minimum latency,
- maximum latency,
- 95th-percentile latency.

These measurements provide both a central estimate of inference time and an indication of the variability observed during repeated execution.

6.2 Benchmark Methodology

The benchmark was performed using the deployed ONNX model executed through ONNX Runtime.

Before collecting the measurements, 1000 inference executions were performed as a warm-up phase. These executions were not included in the reported benchmark statistics.

After the warm-up phase, 10000 inference executions were measured.

For each measured execution, the same inference interface used by the edge application was exercised. The model receives the 20-dimensional feature representation generated by the feature-extraction stage and returns the predicted class and associated probability information.

The benchmark therefore evaluates the computational cost of the deployed inference stage rather than the complete end-to-end system latency.

The measurement procedure can be summarized as:

> ONNX Model Loading → Warm-Up → Measured Inference → Latency Statistics

The separation between warm-up and measurement reduces the influence of initial execution effects on the reported latency statistics.

6.3 Benchmark Configuration

The benchmark configuration is summarized in Table 6.1.

Table 6.1: Configuration of the Edge AI inference benchmark.

Parameter	Value
Inference engine	ONNX Runtime
Model	Random Forest exported to ONNX
Input dimension	20 features
Warm-up executions	1000
Measured executions	10000
Input type	32-bit floating point

The benchmark therefore uses the same model representation and input dimensionality as the deployed Edge AI pipeline.

6.4 Measured Inference Latency

The measured latency statistics are presented in Table 6.2.

Table 6.2: Measured ONNX Runtime inference latency.

Metric	Measured value
Mean	0.014 197 ms
Median	0.012 600 ms
Standard deviation	0.010 464 ms
Minimum	0.009 000 ms
Maximum	0.415 100 ms
P95	0.023 400 ms

The mean inference latency is approximately 0.014 197 ms, corresponding to approximately 14.197 µs.

The median latency is approximately 12.6 µs, while the 95th-percentile latency is approximately 23.4 µs.

The maximum measured value reaches approximately 0.4151 ms. This value is substantially higher than the median and mean and indicates the presence of occasional latency outliers during the repeated benchmark.

The P95 value provides a useful characterization of the upper range of the majority of measured executions, while the maximum represents the largest individual value observed during the benchmark.

6.5 Interpretation of the Measured Latency

The measured mean latency of 0.014 197 ms demonstrates that the deployed Random Forest model requires a small amount of computational time for a single inference under the benchmark conditions.

The P95 latency of 0.023 400 ms indicates that 95% of the measured inference executions completed within approximately 23.4 µs under the benchmark conditions.

The maximum latency of 0.415 100 ms should not be interpreted as the typical inference time. Instead, it represents the largest value observed during the 10000 measured executions.

This distinction is important because real-time systems must consider both typical execution time and execution-time variability.

The benchmark therefore provides three complementary observations:

1. the mean and median describe the central computational cost of inference;

2. the P95 value characterizes the upper range of the majority of measured executions;

3. the maximum value identifies the largest execution-time outlier observed during the benchmark.

6.6 Edge Versus Simulated Cloud Reference

To provide a system-level reference, the measured Edge AI inference latency was compared with a simulated cloud-processing reference used in the project.

The simulated cloud reference corresponds to a processing latency of approximately 25 ms.

Using the measured mean edge latency, the ratio between the simulated cloud reference and the measured edge inference latency is:

$$R = \frac{25}{0.014197} \approx 1760.69 \approx 1761.$$

Thus, the simulated cloud-processing reference is approximately 1761 times higher in latency than the measured mean Edge AI inference latency.

This comparison should be interpreted as a benchmark ratio rather than a direct measurement of a real cloud service. The 25 ms value represents the simulated reference used in the project and does not characterize the latency of a particular production cloud platform.

The comparison nevertheless illustrates the potential computational advantage of performing the classification locally rather than introducing an additional remote processing stage.

6.7 Latency and Real-Time Processing Context

The inference latency must be interpreted in relation to the structure of the complete processing pipeline.

Each signal window contains 256 samples acquired at a sampling frequency of 1 MHz, corresponding to an observation duration of:

$$T_{\text{window}} = \frac{256}{1 \times 10^6} = 256\,\mu\text{s}.$$

The measured mean inference time of approximately 14.197 µs is substantially smaller than the duration represented by one signal window.

Similarly, the P95 latency of approximately 23.4 µs remains below the 256 µs observation-window duration.

This comparison indicates that the computational inference stage represents a relatively small processing interval compared with the duration represented by each feature-extraction window.

However, this observation should not be interpreted as proof that the complete system has an end-to-end latency of less than 256 µs. The complete processing chain also contains signal acquisition, FPGA processing, feature extraction, software scheduling, communication, and monitoring operations.

The present benchmark evaluates only the ONNX Runtime inference stage.

6.8 Relationship Between Inference and Communication

The inference benchmark should also be distinguished from the communication latency of the MQTT and monitoring infrastructure.

The measured inference latency represents the execution time required by ONNX Runtime to process a feature vector and generate the model output.

It does not include:

- MQTT transmission time,
- network scheduling,
- FastAPI processing,
- InfluxDB storage,
- Grafana visualization,
- Digital Twin update overhead.

Consequently, the measured mean of 0.014 197 ms should be interpreted as the computational inference latency of the Edge AI model rather than as the end-to-end system response time.

This separation is important because computational inference and communication represent distinct performance components of the proposed architecture.

6.9 Benchmark Limitations

The benchmark provides a controlled measurement of ONNX Runtime inference performance, but several limitations must be considered.

First, the measurements characterize the execution environment used for the experiment. Inference latency can vary with processor architecture, operating-system scheduling, background processes, runtime configuration, and other environmental factors.

Second, the benchmark evaluates repeated model inference using the deployed ONNX representation. It does not measure the complete physical-to-cloud or physical-to-dashboard response time.

Third, the maximum observed latency is affected by occasional execution outliers. The P95 value is therefore useful for characterizing the upper range of the majority of executions, while the maximum should be interpreted as an observed maximum rather than as a guaranteed deterministic worst-case bound.

Finally, the simulated cloud comparison is a reference scenario rather than a measurement of a specific external cloud infrastructure.

These limitations do not invalidate the measured inference results, but they define the scope within which the latency values should be interpreted.

6.10 Performance Summary

The Edge AI benchmark demonstrates that the deployed Random Forest model can be executed with low computational latency through ONNX Runtime under the benchmark conditions.

Across 10000 measured executions following 1000 warm-up executions, the model achieved a mean latency of 0.014 197 ms, a median latency of 0.012 600 ms, and a P95 latency of 0.023 400 ms.

The maximum observed latency was 0.415 100 ms, indicating occasional execution-time outliers.

Relative to the simulated 25 ms cloud-processing reference, the simulated reference latency is approximately 1761 times higher than the measured mean edge inference latency.

The benchmark therefore supports the use of local inference as a low-latency computational stage within the proposed architecture. The result complements the classification performance presented in Chapter 5 and provides the computational basis for the subsequent CPS and IIoT integration discussed in Chapters 7 and 8.

Chapter 7

Cyber-Physical System Integration

7.1 CPS Integration Objectives

The objective of this chapter is to integrate the deterministic signal-processing and Edge AI stages into a Cyber-Physical System (CPS)-oriented architecture.

The preceding chapters focused on the processing of the acquired signal, the FPGA-based detection mechanism, feature extraction, classification, and inference performance. The purpose of the CPS layer is to transform these processing results into a persistent representation of the monitored cable harness condition.

In the proposed architecture, the CPS layer provides an intermediate state representation between the physical processing pipeline and the higher-level monitoring infrastructure.

The main objectives of this integration are:

- maintain a software representation of the monitored cable harness;
- update the system state after each classified event;
- maintain a health score representing the current inferred condition;
- associate classification results with FPGA-generated information;
- provide a consistent state representation for subsequent communication and monitoring;
- validate the state-update mechanism using representative event scenarios.

The resulting architecture is intentionally lightweight. It is therefore considered a CPS-oriented implementation with a lightweight Digital Twin rather than a complete industrial digital-twin platform.

7.2 CPS-Oriented Architecture

The proposed CPS architecture connects the physical signal-processing pipeline with a software representation of the monitored system.

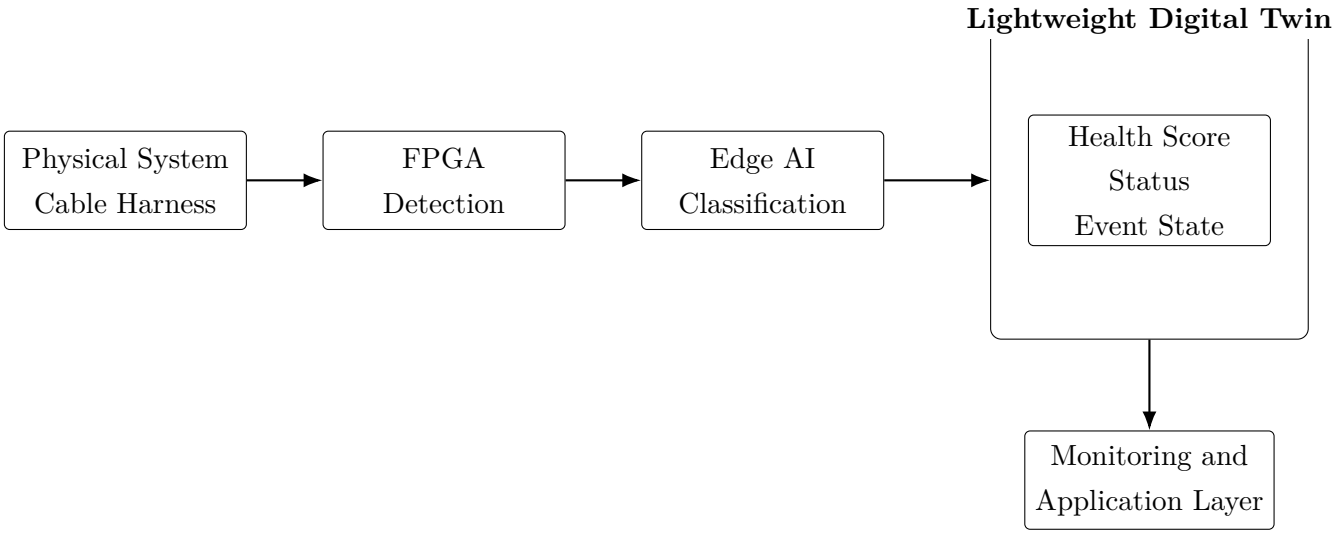


Figure 7.1: CPS-oriented integration of the physical processing pipeline, Edge AI, and lightweight Digital Twin.

Figure 7.1 illustrates the relationship between the physical processing stages and the software representation of the monitored system.

The physical cable harness generates the signal that is processed by the FPGA. The FPGA produces deterministic detection information, which is subsequently combined with signal-derived features and provided to the Edge AI classifier.

The classification result is then supplied to the Digital Twin. Instead of directly representing the raw signal, the Digital Twin maintains a higher-level state describing the inferred condition of the monitored cable harness.

The resulting state can subsequently be exposed to monitoring and application services.

7.3 Digital Twin Representation

The Digital Twin is implemented as a lightweight software object associated with the monitored device.

The implementation is based on the class:

```
DigitalTwin(device_id)
```

The object maintains the current state of the monitored system and updates this state whenever a new classified event is received.

The main state variables are:

- device identifier;
- health score;
- operating status;
- last detected event;
- last classification confidence;
- total number of events;
- number of FPGA-triggered events.

The initial state of the implemented device is:

- device identifier: `CABLE_TEST_01`;
- health score: 100%;
- status: `NORMAL`;
- last event: `normal`;
- confidence: 0;
- total events: 0;
- FPGA events: 0.

This state provides a compact representation of the current condition of the monitored system.

7.4 Digital Twin State Model

The information maintained by the Digital Twin can be organized into several categories.

The device identity is represented by the device identifier. This allows the state to be associated with a specific monitored system.

The condition of the device is represented primarily by the health score and operating status. The health score provides a numerical representation of the current inferred condition, while the status provides a higher-level state classification.

The most recent classification result is represented by the last detected event and its associated confidence.

Finally, cumulative counters provide information about the history of the monitored system during the current execution.

The relationship between these state variables is illustrated in Figure 7.2.

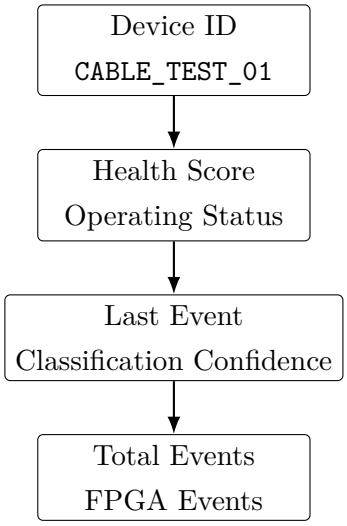


Figure 7.2: Main state variables maintained by the lightweight Digital Twin.

The state representation is intentionally compact. It does not attempt to reproduce the complete physical behavior of the cable harness.

Instead, it provides a software representation of the condition inferred by the processing pipeline.

7.5 Digital Twin Update Mechanism

The Digital Twin is updated through the method:

```
twin.update(event_type, confidence, fpga_info)
```

The method receives the classified event type, the classification confidence, and the FPGA-derived information associated with the observation.

The update operation modifies the state of the Digital Twin according to the new observation.

The update process can be summarized as:

1. receive the classified event;
2. receive the corresponding confidence value;
3. process the FPGA event information;
4. update the health score;
5. update the operating status;
6. store the most recent event;

7. update the classification confidence;

8. increment the corresponding event counters.

This mechanism provides a direct interface between the Edge AI inference stage and the higher-level CPS representation.

7.6 CPS Data Flow

The complete CPS data flow begins with the signal generated by the physical cable harness.

The FPGA performs deterministic processing and generates event information when the configured detection criterion is satisfied.

The signal window and FPGA information are then processed by the feature-extraction stage. The resulting 20-dimensional feature vector is provided to the Edge AI classifier.

The classifier produces a predicted class and associated confidence information. These results are passed to the Digital Twin together with the FPGA-derived event information.

The Digital Twin updates its internal state and produces the current health score and operating status.

The resulting state can then be transmitted to the monitoring infrastructure.

The complete sequence is therefore:

Physical Signal → FPGA Detection → Feature Extraction → Edge AI Classification → Digital Twin Update → System State

This sequence is illustrated conceptually in Figure 7.3.

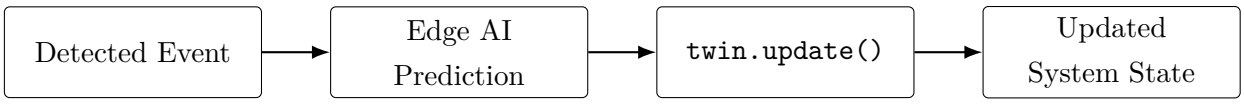


Figure 7.3: Digital Twin update sequence following an Edge AI classification result.

The Digital Twin therefore acts as a state-management layer between the inference results and the monitoring services.

7.7 Health Score and Operating Status

The Digital Twin maintains a numerical health score that evolves as new observations are received.

The health score provides a compact representation of the accumulated condition of the monitored cable harness.

The operating status provides a categorical interpretation of the current state. In the implemented tests, the status is updated together with the health score.

The use of both a numerical score and a categorical status provides two complementary representations.

The numerical health score allows changes in system condition to be observed quantitatively, while the categorical status provides a simpler interpretation for monitoring applications.

The health score should therefore be interpreted as an application-level state indicator generated by the implemented Digital Twin logic. It should not be interpreted as a physical measurement of cable integrity.

7.8 Digital Twin Validation

The Digital Twin implementation was validated using representative event sequences.

Three principal test cases were used to verify the state-update mechanism.

The first test corresponds to a normal event with a classification confidence of 99%. The resulting Digital Twin state maintained a health score of 100% and a `NORMAL` status.

The second test corresponds to a micro-cut event with a classification confidence of 98%. After the update, the Digital Twin reported a health score of 98.6% and retained a `NORMAL` status.

The third test corresponds to an abrupt event with a classification confidence of 97% and associated FPGA information. After the update, the health score became 95.7%, the status remained `NORMAL`, the total event count reached 3, and the FPGA event counter became 1.

The validation results are summarized in Table 7.1.

Table 7.1: Validation results of the Digital Twin state-update mechanism.

Event	Confidence	Health	Status	Total Events	FPGA Events
normal	99%	100.0%	NORMAL	1	0
micro-cut	98%	98.6%	NORMAL	2	0
abrupt	97%	95.7%	NORMAL	3	1

The results confirm that the Digital Twin correctly updates its state after receiving successive event observations.

In particular, the third test confirms that FPGA-derived information is also incorporated into the state representation, as indicated by the FPGA event counter increasing from zero to one.

7.9 Relationship Between FPGA, Edge AI, and Digital Twin

The CPS-oriented architecture establishes a hierarchical relationship between the FPGA detector, Edge AI classifier, and Digital Twin.

The FPGA operates at the lowest computational level and performs deterministic signal processing and event detection.

The Edge AI stage operates at a higher level by interpreting the extracted signal characteristics and assigning the observation to one of the predefined classes.

The Digital Twin operates at the system-state level. It does not replace the FPGA detector or the machine-learning classifier. Instead, it aggregates their outputs into a persistent representation of the monitored device condition.

This organization separates three distinct functions:

- deterministic signal-level detection;
- data-driven event classification;
- system-level state representation.

Such separation is useful because each layer has a different responsibility and can therefore be evaluated independently.

7.10 CPS Integration with the Monitoring Infrastructure

The Digital Twin is not an isolated software component. Its state is subsequently used by the monitoring infrastructure described in Chapter 8.

After each event update, the resulting health score, operating status, event information, and FPGA-related information can be included in the messages transmitted by the edge application.

MQTT provides the communication interface between the edge processing pipeline and the monitoring services.

The FastAPI service receives the corresponding messages and exposes the current system information through REST endpoints.

InfluxDB and Grafana subsequently provide persistent storage and visualization of the system state.

The resulting information flow is:

$$\text{Digital Twin State} \rightarrow \text{MQTT} \rightarrow \text{FastAPI / InfluxDB} \rightarrow \text{Grafana}$$

The communication and monitoring implementation is treated separately in Chapter 8 in order to distinguish CPS state management from the Industrial IoT infrastructure used to expose and visualize that state.

7.11 CPS Implementation Scope

The implemented Digital Twin provides a lightweight software representation of the monitored cable harness.

Its purpose is to maintain the current inferred condition of the system and provide a structured state representation for the monitoring infrastructure.

The implementation does not attempt to model the complete physical behavior of the cable harness, electrical parameters, degradation mechanisms, or mechanical characteristics.

Furthermore, the current Digital Twin does not perform feedback control of the physical system. Information flows from the physical processing pipeline toward the software representation and monitoring layer.

Consequently, the implemented architecture should be classified as a CPS-oriented monitoring architecture with a lightweight Digital Twin.

This scope is appropriate for the objective of integrating FPGA detection, Edge AI classification, and system-level monitoring without introducing unnecessary complexity into the prototype.

7.12 Chapter Summary

This chapter presented the integration of the FPGA and Edge AI processing stages into a CPS-oriented architecture.

A lightweight Digital Twin was implemented to maintain the current state of the monitored cable harness. The state includes the device identifier, health score, operating status, last detected event, classification confidence, total event count, and FPGA event count.

The Digital Twin is updated through the `twin.update()` mechanism after each classified observation.

Validation using normal, micro-cut, and abrupt events demonstrated that the Digital Twin correctly updates its state and incorporates FPGA-derived event information.

The CPS layer therefore provides a system-level representation between the deterministic signal-processing and machine-learning stages and the Industrial IoT monitoring infrastructure.

The MQTT communication, REST interface, time-series storage, and Grafana visualization associated with this state representation are discussed in the following

chapter.

Chapter 8

Industrial IoT and Monitoring Infrastructure

8.1 Chapter Overview

The proposed cable harness test bench extends the FPGA-based detection and Edge AI classification stages with an Industrial Internet of Things (IIoT) monitoring infrastructure. This layer provides communication, state management, time-series data storage, REST-based access, visualization, and containerized deployment.

The resulting infrastructure connects the embedded detection and artificial intelligence components to higher-level monitoring services. MQTT is used as the communication layer, FastAPI provides a REST interface, the Digital Twin maintains the current system state, InfluxDB stores time-series measurements, and Grafana provides monitoring dashboards. Docker Compose integrates the different software services into a reproducible execution environment.

The complete infrastructure was validated through an end-to-end experiment using a controlled sequence of 20 events. The sequence contained four events for each of the five classes: `normal`, `micro_cut`, `noise`, `transient`, and `abrupt`. The final health score reached 75.2%, with six FPGA detections and a final system status of `WARNING`.

8.2 IIoT System Architecture

The overall monitoring architecture is organized into several layers. The FPGA performs deterministic signal processing and event detection, while the Edge AI layer performs event classification using the trained Random Forest model deployed through ONNX Runtime. The resulting event information is published through MQTT.

The MQTT broker distributes the information to the monitoring services. FastAPI provides programmatic access to the system state and event history, while the Digital

Twin maintains a software representation of the current cable condition. In parallel, event data are stored in InfluxDB and exposed through Grafana for time-series monitoring.

Figure 8.1 summarizes the complete architecture.

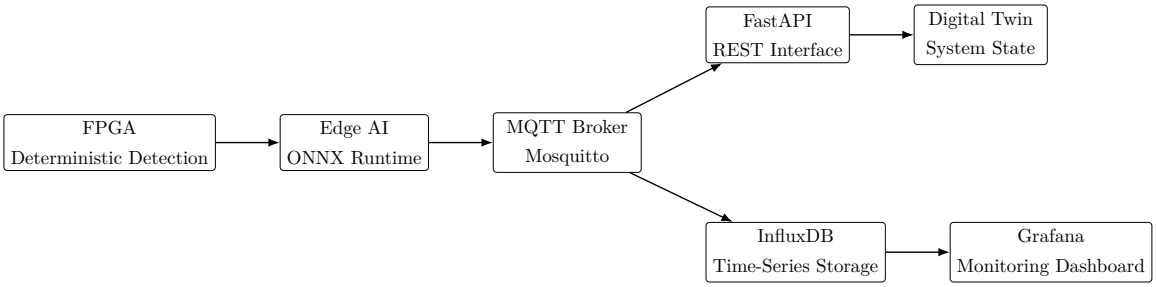


Figure 8.1: Overall IIoT and monitoring architecture of the cable harness test bench.

The architecture separates deterministic embedded processing from higher-level monitoring functions. This separation allows the FPGA and Edge AI stages to operate independently from the visualization and data storage services.

8.3 MQTT-Based Communication Layer

MQTT was selected as the communication protocol between the event-generation layer and the monitoring infrastructure. The publish/subscribe model decouples the Edge AI publisher from the services consuming the event data.

The deployed broker is Eclipse Mosquitto. The local implementation uses port 1883 for MQTT communication and port 9001 for MQTT over WebSockets.

The main topics are:

- `cable/01/events`: event and Edge AI information;
- `cable/01/status`: current device status information;
- `cable/01/diagnostics`: diagnostic information.

Figure 8.2 illustrates the communication paths.

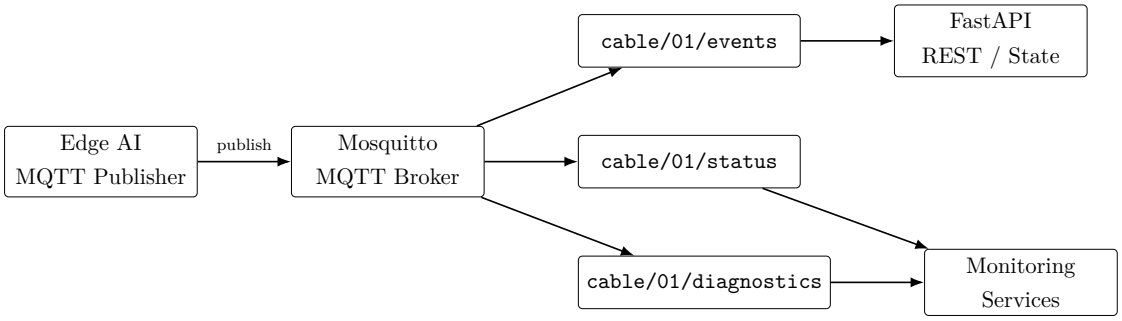


Figure 8.2: MQTT communication architecture and deployed topics.

The event messages contain the information required by the monitoring pipeline. The principal fields include the device identifier, event type, classification confidence, FPGA detection information, health score, signal statistics, sampling parameters, and timestamp.

Table 8.1: Main information transmitted by the event messages.

Field	Description
`device_id`	Identifier of the monitored cable system.
`event_type`	Classified event type.
`confidence`	Edge AI classification confidence.
`fpga_triggered`	Indicates whether the FPGA detector was triggered.
`fpga_event_count`	Number of FPGA detections associated with the event.
`health_score`	Current Digital Twin health score.
`signal_rms`	RMS value of the analyzed signal window.
`sample_rate_hz`	Sampling frequency of the signal.
`window_size`	Number of samples in the analysis window.
`feature_count`	Number of features supplied to the AI model.
`timestamp_us`	Event timestamp information.

8.4 Time-Series Data Storage with InfluxDB

InfluxDB 2.x was used as the time-series database for the monitoring infrastructure. Event information is stored in the `cable_harness` bucket using the `cable_events` measurement.

The database organization separates tags, which are useful for filtering and grouping, from numerical fields representing measured or computed values.

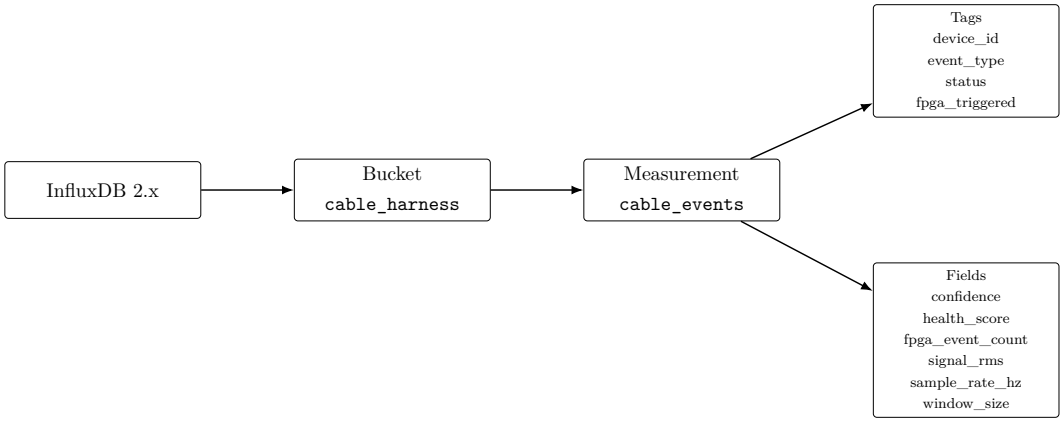


Figure 8.3: InfluxDB organization used for cable event time-series storage.

Table 8.2: InfluxDB storage configuration.

Parameter	Configuration
Database	InfluxDB 2.x
Organization	`cable-harness`
Bucket	`cable_harness`
Measurement	`cable_events`
Timestamp	UTC timestamp
Retention	Infinite for the project bucket

The event records stored in InfluxDB provide the data source used by the Grafana monitoring dashboards.

8.5 REST Monitoring Interface with FastAPI

FastAPI provides a lightweight REST interface for programmatic monitoring of the CPS. It exposes the current system state, event history, Edge AI predictions, and diagnostic information.

The implemented endpoints are summarized in Table 8.3.

Table 8.3: Implemented FastAPI REST endpoints.

Method	Endpoint	Function
`GET`	/	System availability check.
`GET`	/status	Current device state and health information.
`GET`	/events	Stored event history.
`GET`	/predictions	Edge AI prediction history.
`GET`	/diagnostics	Diagnostic information.
`POST`	/test-event	Test and integration event injection.

The FastAPI service was validated through its automatically generated Swagger interface. Figure 8.4 should contain the real Swagger screenshot captured from the implemented application.

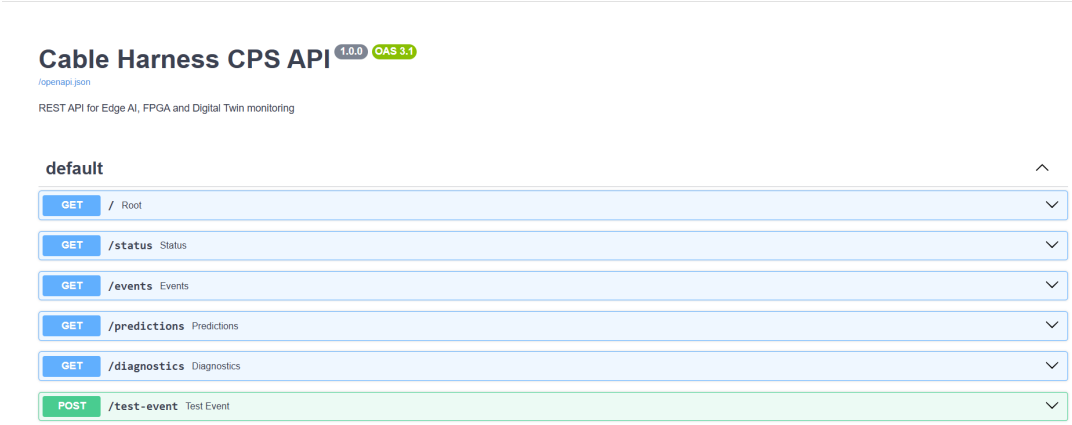


Figure 8.4: FastAPI REST monitoring interface and implemented endpoints.

The root endpoint returned the following system-level result:

```
{
  "system": "Cable Harness CPS",
  "status": "online"
}
```

During the final validation state, the `/status` endpoint returned:

```
{
  "device_id": "CABLE_TEST_01",
  "status": "online",
  "health_score": 75.19999999999999,
  "device_status": "TEST",
  "total_events": 41,
  "fpga_events": 12,
  "last_event": "micro_cut",
  "last_confidence": 0.98
}
```

The value of 41 events corresponds to two controlled 20-event publisher runs plus one additional API test event. Therefore, this value represents the accumulated in-memory FastAPI history rather than the single controlled 20-event validation experiment.

The `/events` endpoint contained 40 publisher events, corresponding to the two completed 20-event runs. The `/predictions` endpoint likewise contained 40 Edge AI predictions. The `/diagnostics` endpoint reported zero diagnostic messages during the validation.

8.6 Digital Twin and System State Representation

A lightweight Digital Twin was implemented to maintain a software representation of the current cable condition. It does not attempt to replicate the complete physical system. Instead, it maintains the principal state variables required for monitoring and condition assessment.

The Digital Twin state includes the device identifier, health score, current status, last detected event, classification confidence, total event count, and FPGA event count.

The state update mechanism is illustrated in Figure 8.5.

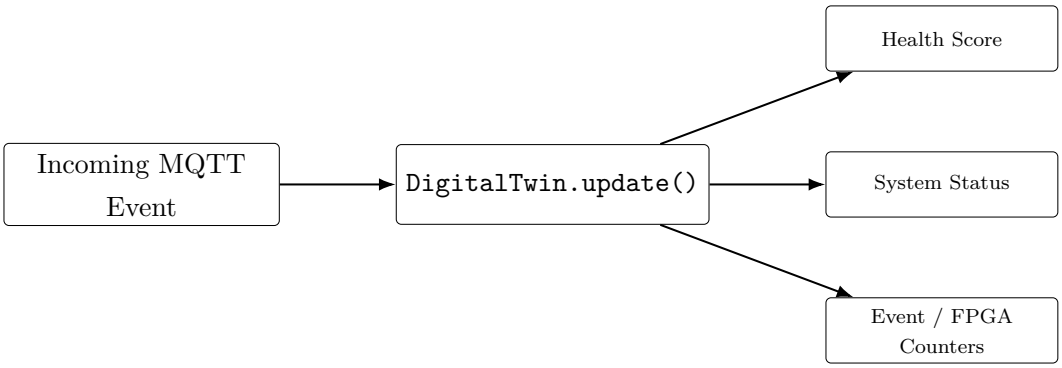


Figure 8.5: Digital Twin state update mechanism.

The Digital Twin was tested independently before integration. A normal event with 99% confidence resulted in a health score of 100% and a `NORMAL` status. A micro-cut event with 98% confidence reduced the health score to 98.6%. An FPGA-triggered abrupt event with 97% confidence resulted in a health score of 95.7% and increased the FPGA event counter.

For the controlled 20-event validation, the Digital Twin reached a final health score of 75.2% and a `WARNING` status.

8.7 Grafana Monitoring Dashboard

Grafana was used as the visualization layer for the time-series information stored in InfluxDB. Six monitoring panels were implemented:

1. **Cable Health Score**;
2. **Detected Events**;
3. **Cable Status / Current Health Score**;
4. **FPGA Detections**;
5. **Health Score Evolution**;

6. **Events by Type**.

To keep the report readable, the dashboard is documented using two screenshots rather than one screenshot per panel. The first screenshot contains two panels, while the second contains the remaining four panels.

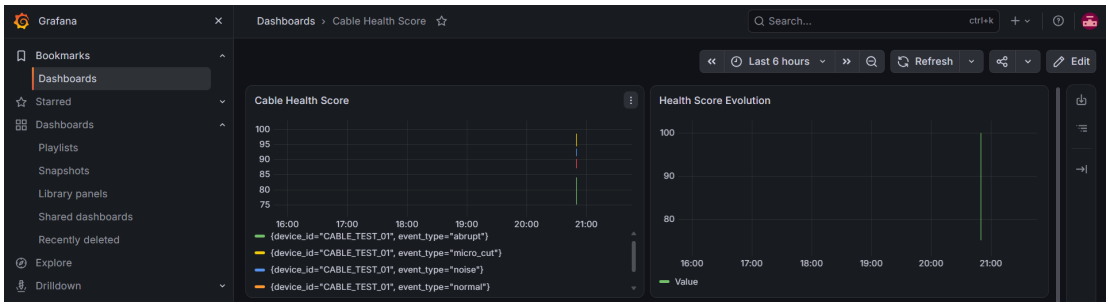


Figure 8.6: Grafana monitoring dashboard: Cable Health Score and Detected Events panels.

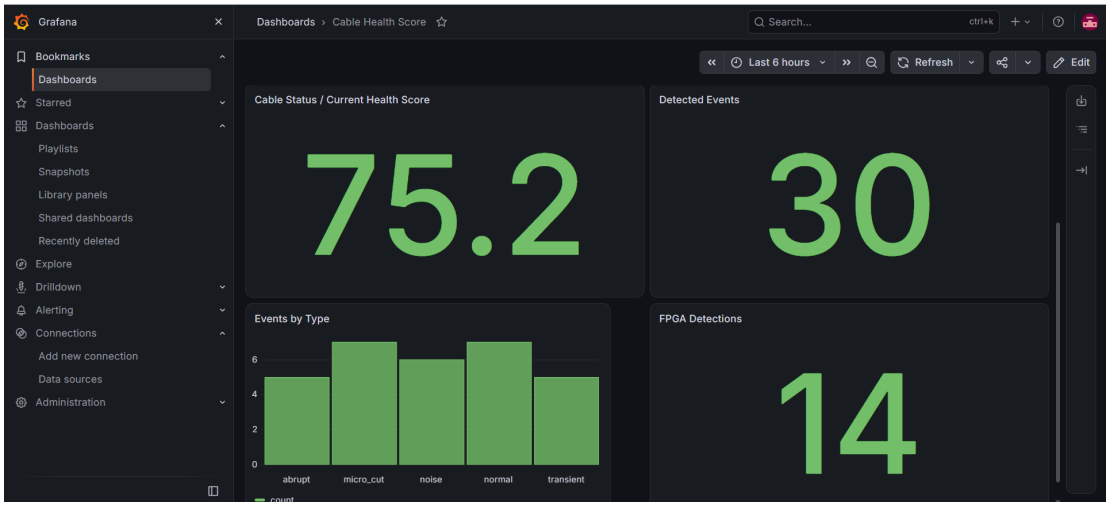


Figure 8.7: Grafana monitoring dashboard: current health, FPGA detections, health evolution, and event distribution panels.

The controlled 20-event validation produced the following health-score sequence:

Table 8.4: Health score evolution during the controlled 20-event run.

Event	Health	Event	Health
1	100.0	11	92.0
2	100.0	12	91.2
3	100.0	13	90.2
4	100.0	14	89.2
5	98.6	15	88.2
6	97.2	16	87.2
7	95.8	17	84.2
8	94.4	18	81.2
9	93.6	19	78.2
10	92.8	20	75.2

The final health score was therefore:

$$\boxed{\text{Health Score}_{final} = 75.2\%}$$

with the final controlled system status equal to `WARNING`.

8.7.1 Grafana Panel Results

Cable Health Score

The *Cable Health Score* time-series panel displays the evolution of the Digital Twin health score over the event sequence. The 20 controlled values were:

```
100.0, 100.0, 100.0, 100.0,
98.6, 97.2, 95.8, 94.4,
93.6, 92.8, 92.0, 91.2,
90.2, 89.2, 88.2, 87.2,
84.2, 81.2, 78.2, 75.2
```

The graph therefore shows a progressive degradation of the simulated cable health condition from 100% to 75.2%.

Detected Events

The *Detected Events* panel counts event records stored in the `cable_events` measurement during the selected Grafana time range.

For the controlled experiment, the publisher generated 20 events. However, the live Grafana database retained previous events because the configured query used a one-hour

time range. Consequently, the dashboard displayed larger accumulated counts in some captures. These values reflect the database history and should not be interpreted as the number of events in the single controlled run.

Cable Status / Current Health Score

The *Cable Status / Current Health Score* panel displays the most recent health-score value. For the controlled validation, the latest value was:

75.2%

This panel represents the current state, whereas the *Cable Health Score* panel provides the historical evolution.

FPGA Detections

The *FPGA Detections* panel aggregates the `fpga_event_count` field.

For the controlled 20-event run:

6 FPGA detections

The six detections resulted from:

- four `noise` events;
- two `abrupt` events.

As with the event-count panel, a live Grafana query over historical database records can display a larger accumulated value.

Health Score Evolution

The *Health Score Evolution* panel provides a chronological time-series representation of the health score. For the controlled run, it contains the same 20 health-score observations listed in Table 8.4, ending at 75.2%.

Events by Type

The *Events by Type* panel groups event records according to the `event_type` tag.

For the controlled 20-event sequence, the intended distribution was:

Table 8.5: Event distribution in the controlled validation sequence.

Event Type	Number of Events
`normal`	4
`micro_cut`	4
`noise`	4
`transient`	4
`abrupt`	4
Total	**20**

This controlled distribution is different from a live Grafana count if older records remain inside the selected time range.

8.8 Docker-Based Deployment Infrastructure

Docker Compose was used to integrate the software components into a single containerized infrastructure. The persistent services are Mosquitto, InfluxDB, Grafana, and FastAPI. The Edge AI publisher is also containerized and can be executed as part of the project stack.

The deployment uses a dedicated Docker network named `cable-network`.

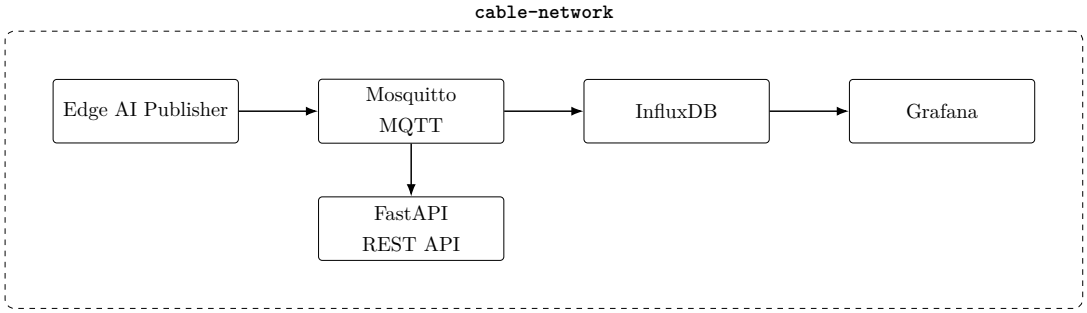


Figure 8.8: Docker Compose deployment architecture and service network.

The principal exposed ports are summarized below.

Table 8.6: Main Docker service ports.

Service	Container	Port
FastAPI	`cable-api`	8000
Grafana	`cable-grafana`	3000
InfluxDB	`cable-influxdb`	8086
Mosquitto MQTT	`cable-mosquitto`	1883
Mosquitto WebSocket	`cable-mosquitto`	9001

The deployment was verified using the Docker Compose service listing. Figure 8.9 corresponds to the actual terminal output obtained from the implemented system.

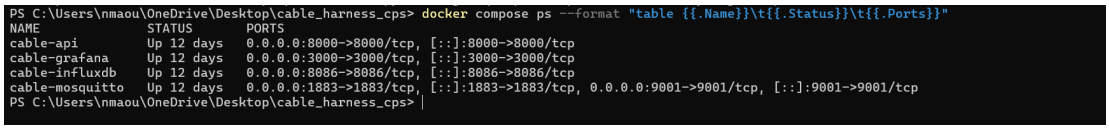


Figure 8.9: Docker Compose running services and exposed communication ports.

The captured deployment state showed the following active services:

Table 8.7: Observed Docker Compose service status.

Container	Status	Ports
`cable-api`	Up 12 days	8000
`cable-grafana`	Up 12 days	3000
`cable-influxdb`	Up 12 days	8086
`cable-mosquitto`	Up 12 days	1883, 9001

The four persistent services were therefore running continuously at the time of capture. The Edge AI publisher is an execution-oriented service and does not necessarily remain visible in the running-container list after its publishing process terminates.

8.9 End-to-End Monitoring Validation

The complete infrastructure was validated by executing the Edge AI publisher and following the resulting event information through the monitoring stack.

The validated data path was:

Edge AI → MQTT → FastAPI / Digital Twin → InfluxDB → Grafana

The controlled validation consisted of 20 events, with four events assigned to each event class. Six of these events produced FPGA detections.

Table 8.8: End-to-end monitoring validation results.

Parameter	Result
Controlled events	20
Event classes	5
Events per class	4
FPGA detections	6
Initial health score	100%
Final health score	75.2%
Final controlled status	`WARNING`
FastAPI	Operational
MQTT broker	Operational
InfluxDB	Operational
Grafana	Operational
Digital Twin	Operational
Docker Compose	Operational

The validation demonstrates that an event generated by the Edge AI layer can be transported through MQTT, processed by the monitoring services, reflected in the Digital Twin state, stored as time-series data, and visualized in Grafana.

It is important to distinguish the controlled experiment from accumulated dashboard and FastAPI history. The controlled experiment contains exactly 20 events and six FPGA detections. The FastAPI history later contained 40 publisher events because the 20-event sequence was executed twice, followed by one additional API test event. Similarly, Grafana can display accumulated records when its selected time range includes previous runs.

8.10 Chapter Summary

This chapter presented the Industrial IoT and monitoring infrastructure developed around the FPGA and Edge AI cable harness test bench. MQTT provided the communication layer, FastAPI exposed a REST monitoring interface, and a lightweight Digital Twin maintained the current system state. InfluxDB provided time-series storage, while Grafana enabled real-time-oriented visualization of cable health, event counts, FPGA detections, and event distribution.

The complete software infrastructure was integrated using Docker Compose. The end-to-end validation confirmed the operation of the communication, monitoring, storage, visualization, and state-management components. For the controlled 20-event experiment, the system reached a final health score of 75.2% and recorded six FPGA

detections, demonstrating the integration of embedded detection, Edge AI, and IIoT monitoring into a unified cyber-physical architecture.

Note. The Grafana dashboard may display values such as 30 detected events and 14 FPGA detections in the presented figures. These values include historical records retained in the InfluxDB database from previous executions. Therefore, they do not correspond exclusively to the controlled 20-event experiment reported in this chapter.

Chapter 9

Dockerized Deployment and End-to-End Validation

9.1 Chapter Overview

The complete cable harness monitoring system was deployed as a containerized software infrastructure using Docker Compose. The deployment integrates the MQTT broker, FastAPI service, InfluxDB database, Grafana visualization platform, and the Edge AI publisher within a common execution environment.

The objective of this chapter is to document the containerized deployment, verify the operation of the individual services, and validate the complete data path from Edge AI event generation to monitoring and visualization.

The final validation was performed using a controlled sequence of 20 events containing four events from each of the five classes: `normal`, `micro_cut`, `noise`, `transient`, and `abrupt`. The controlled experiment produced six FPGA detections and resulted in a final Digital Twin health score of 75.2%.

9.2 Docker Compose Deployment

Docker Compose was used to define and coordinate the software services required by the monitoring infrastructure. The deployment provides a reproducible environment in which the communication, storage, API, visualization, and Edge AI components can operate together.

The main services used in the project are:

- **Mosquitto**: MQTT broker responsible for event communication;
- **FastAPI**: REST interface and system monitoring service;
- **InfluxDB**: time-series database used to store event and system measurements;

- **Grafana**: visualization and monitoring platform;
- **Edge AI Publisher**: application responsible for generating and publishing classified events.

The persistent monitoring services communicate through the dedicated Docker network `cable-network`.

9.3 Containerized Service Architecture

The Dockerized architecture separates the individual services while allowing them to communicate through the internal Docker network.

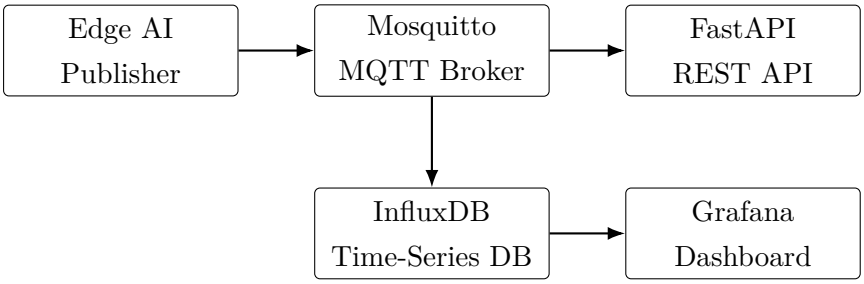


Figure 9.1: Containerized software architecture of the cable harness monitoring system.

The Edge AI publisher sends event information to the MQTT broker. FastAPI consumes the relevant information and provides programmatic access to the current system state and event history. In parallel, event measurements are stored in InfluxDB and subsequently visualized through Grafana.

9.4 Docker Network and Service Ports

The services communicate internally through the Docker network `cable-network`. The principal externally exposed ports are summarized in Table 9.1.

Table 9.1: Docker services and exposed ports.

Service	Container	Port
FastAPI	`cable-api`	8000
Grafana	`cable-grafana`	3000
InfluxDB	`cable-influxdb`	8086
Mosquitto MQTT	`cable-mosquitto`	1883
Mosquitto WebSocket	`cable-mosquitto`	9001

Port 1883 is used for standard MQTT communication, while port 9001 provides MQTT over WebSockets. FastAPI, InfluxDB, and Grafana expose their respective HTTP interfaces through ports 8000, 8086, and 3000.

9.5 Running Container Validation

The deployment was verified using the Docker Compose service listing. The command

```
docker compose ps --format "table {{.Name}}\t{{.Status}}\t{{.Ports}}"
```

was used to inspect the active containers.

The captured deployment contained the following persistent services:

Table 9.2: Observed Docker Compose service status.

Container	Status	Ports
`cable-api`	Up 12 days	8000
`cable-grafana`	Up 12 days	3000
`cable-influxdb`	Up 12 days	8086
`cable-mosquitto`	Up 12 days	1883, 9001

The Edge AI publisher is execution-oriented. Unlike the persistent monitoring services, it can terminate after completing its publishing sequence and therefore does not necessarily remain visible in the running-container list.

9.6 FastAPI Service Validation

The FastAPI service was validated through its REST interface and automatically generated Swagger documentation.

The implemented endpoints are:

Table 9.3: FastAPI endpoints validated during deployment.

Method	Endpoint	Function
`GET`	/	System availability check.
`GET`	/status	Current device state and health information.
`GET`	/events	Stored event history.
`GET`	/predictions	Edge AI prediction history.
`GET`	/diagnostics	Diagnostic information.
`POST`	/test-event	Test and integration event injection.

The root endpoint returned:

```
{
  "system": "Cable Harness CPS",
  "status": "online"
}
```


The `/diagnostics` endpoint reported zero diagnostic messages during the validation.

The final captured `/status` response contained a health score of approximately 75.2%. The accumulated FastAPI history contained 41 events: 40 events generated by two controlled 20-event publisher runs and one additional API test event.

9.7 MQTT Communication Validation

The MQTT communication layer was validated by executing the Edge AI publisher and observing the transfer of event information through the Mosquitto broker.

The validated communication path was:

Edge AI Publisher → Mosquitto → Monitoring Services

The publisher generated event messages containing the event class, classification confidence, FPGA information, health information, signal statistics, and other monitoring parameters.

The MQTT broker remained active as a persistent service throughout the validation.

9.8 InfluxDB Data Storage Validation

InfluxDB was used to persist the monitoring data generated during the experiment.

The project uses the following database organization:

- Organization: `cable-harness`;
- Bucket: `cable_harness`;
- Measurement: `cable_events`.

The stored records contain tags such as `device_id`, `event_type`, `status`, and `fpga_triggered`. Numerical fields include `confidence`, `health_score`, `fpga_event_count`, `signal_rms`, `sample_rate_hz`, and `window_size`.

The database therefore provides the persistent data source used by the Grafana monitoring dashboards.

9.9 Grafana Monitoring Validation

Grafana was used to visualize the time-series information stored in InfluxDB. Six monitoring panels were implemented:

1. **Cable Health Score**;
2. **Detected Events**;
3. **Cable Status / Current Health Score**;
4. **FPGA Detections**;
5. **Health Score Evolution**;
6. **Events by Type**.

For the controlled 20-event experiment, the health score evolved from 100% to 75.2%.

The controlled sequence was:

```
100.0, 100.0, 100.0, 100.0,
98.6, 97.2, 95.8, 94.4,
93.6, 92.8, 92.0, 91.2,
90.2, 89.2, 88.2, 87.2,
84.2, 81.2, 78.2, 75.2
```

The controlled experiment produced six FPGA detections. These corresponded to four `noise` events and two `abrupt` events.

The Grafana dashboard can also display historical records because the queries use a one-hour time range. Consequently, the live dashboard may contain more records than the single controlled 20-event experiment.

9.10 Digital Twin Validation

The Digital Twin was integrated into the monitoring pipeline to maintain the current software representation of the cable condition.

The state update process receives event information and updates the health score, system status, event counters, and FPGA detection counter.

The Digital Twin was first validated independently using representative events. A normal event with 99% confidence resulted in a health score of 100% and a `NORMAL` status. A micro-cut event with 98% confidence resulted in a health score of 98.6%. An FPGA-triggered abrupt event with 97% confidence resulted in a health score of 95.7%.

During the controlled 20-event experiment, the final health score was 75.2% and the final controlled status was `WARNING`.

9.11 End-to-End Validation

The final validation verified the complete monitoring chain from event generation to visualization.

The validated data path was:

Edge AI → MQTT → FastAPI / Digital Twin → InfluxDB → Grafana

The controlled experiment consisted of 20 events, with four events for each of the five event classes.

Table 9.4: End-to-end deployment and monitoring validation results.

Parameter	**Result**
Controlled events	20
Event classes	5
Events per class	4
FPGA detections	6
Initial health score	100%
Final health score	75.2%
Final controlled status	`WARNING`
MQTT broker	Operational
FastAPI	Operational
InfluxDB	Operational
Grafana	Operational
Digital Twin	Operational
Docker Compose	Operational

The validation confirms that the containerized infrastructure can transport, process, store, and visualize event information generated by the Edge AI layer.

9.12 Deployment Limitations and Observations

The validation also identified an important distinction between a controlled experiment and accumulated monitoring history.

The controlled experiment contained exactly 20 events and six FPGA detections. However, the FastAPI service accumulated 40 publisher events because the 20-event sequence was executed twice, followed by one additional API test event.

Similarly, Grafana queries using a one-hour range can include records from previous executions. Therefore, dashboard counters captured during development should be interpreted as database-history values rather than as isolated measurements of the single controlled experiment.

This distinction does not affect the controlled validation results, which are based on the explicitly defined 20-event sequence.

9.13 Chapter Summary

This chapter presented the Dockerized deployment and end-to-end validation of the cable harness monitoring infrastructure.

Docker Compose integrated the MQTT broker, FastAPI service, InfluxDB, Grafana, and Edge AI publishing components into a common execution environment. The individual services were verified through their respective interfaces and communication paths.

The complete monitoring chain was successfully validated from Edge AI event generation through MQTT communication, Digital Twin state updates, InfluxDB storage, and Grafana visualization.

For the controlled experiment, 20 events were processed, six FPGA detections were recorded, and the Digital Twin reached a final health score of 75.2% with a `WARNING` status. These results confirm the functional integration of the embedded detection, Edge AI, and IIoT monitoring components within the final cyber-physical system.

Chapter 10

Discussion and Limitations

10.1 Chapter Overview

This chapter discusses the main results obtained from the proposed intelligent cable harness test bench and identifies the principal limitations of the current implementation.

The developed system combines FPGA-based deterministic detection, Edge AI classification, Digital Twin state representation, and IIoT monitoring within a unified cyber-physical architecture. The results demonstrate the functional integration of these components while also highlighting several areas that require further development.

10.2 Discussion of the Main Results

The FPGA implementation demonstrated the feasibility of performing deterministic signal-change detection on an Artix-7 device. After timing optimization, the final implementation achieved a positive worst negative slack of 0.093 ns with no failing timing endpoints. The resource utilization remained low, with 1.17% LUT utilization, 0.82% flip-flop utilization, and 0.83% DSP48 utilization.

The Edge AI layer provided a complementary classification mechanism. The Random Forest classifier achieved 94.5% accuracy and a macro F1-score of 0.938 on the scenario-level test set. The inference benchmark also showed a mean execution time of 0.014197 ms on the target software environment.

The IIoT infrastructure successfully transported, stored, and visualized the generated event information. The controlled 20-event experiment resulted in six FPGA detections and a final Digital Twin health score of 75.2%.

10.3 System-Level Discussion

The main contribution of the proposed architecture is the integration of complementary processing layers rather than reliance on a single detection mechanism.

The FPGA layer provides deterministic low-level event detection, while the Edge AI layer performs higher-level event classification. The Digital Twin maintains a software representation of the current cable condition, and the IIoT infrastructure provides persistent storage and monitoring.

This separation allows each component to perform a specific function while remaining connected through standardized communication interfaces.

10.4 Limitations

Several limitations remain in the current prototype.

First, the dataset is generated from predefined scenarios rather than from a large collection of experimentally measured cable-harness faults. Consequently, the reported classification performance should be interpreted as validation of the proposed methodology rather than as a direct estimate of industrial field performance.

Second, the current FPGA implementation focuses on deterministic event detection. The timestamp mechanism remains available at the Simulink/reference level but was not retained as a final external FPGA output.

Third, the current Digital Twin is a lightweight state representation of the cable condition. It does not yet implement a complete industrial digital-twin platform or closed-loop physical control.

Finally, the IIoT monitoring infrastructure was validated locally using Docker Compose. Additional testing would be required to evaluate the system under larger data volumes, longer operating periods, network interruptions, and multiple simultaneous devices.

10.5 Chapter Summary

The results demonstrate the functional feasibility of combining FPGA detection, Edge AI classification, Digital Twin state representation, and IIoT monitoring in a unified cable harness monitoring system.

At the same time, the current implementation remains a prototype. The main limitations concern the dataset, experimental validation, FPGA output integration, Digital Twin scope, and large-scale deployment. These limitations define the main directions for future research.

Chapter 11

Future Research Directions

11.1 Chapter Overview

The current prototype provides a foundation for further research in intelligent cable-harness testing, embedded intelligence, and cyber-physical monitoring.

Future work should focus on improving experimental validation, embedded intelligence, system scalability, and industrial applicability.

11.2 Experimental Dataset Expansion

A major direction is the creation of a larger dataset obtained from physical cable harness experiments.

Future measurements could include different cable types, conductor lengths, load conditions, degradation levels, connector conditions, and environmental disturbances. Such a dataset would allow the Edge AI models to be evaluated under conditions closer to real industrial operation.

11.3 Advanced Edge AI Models

The current Random Forest classifier provides a lightweight classification solution. Future work could investigate alternative models such as gradient-boosting methods, compact neural networks, or other embedded machine-learning approaches.

Model compression, quantization, and hardware-aware optimization could also be investigated to enable more complex models on resource- constrained edge platforms.

11.4 Enhanced FPGA Processing

Future FPGA development could integrate additional signal-processing functions directly into the hardware pipeline.

Possible improvements include more advanced feature extraction, adaptive thresholds, improved event timing information, and additional diagnostic outputs. These developments could further reduce the dependence on software-side preprocessing.

11.5 Digital Twin Development

The current Digital Twin can be extended from a lightweight state representation toward a richer system model.

Future research could incorporate historical degradation information, remaining-useful-life estimation, anomaly trends, and maintenance indicators. The Digital Twin could therefore become a more detailed representation of cable condition and degradation evolution.

11.6 Scalable IIoT Deployment

The current Dockerized infrastructure can also be extended toward multi-device deployment.

Future implementations could support multiple cable-harness test benches, centralized monitoring, device authentication, secure MQTT communication, and distributed data processing. Additional experiments could evaluate system behavior under higher event rates and longer operating periods.

11.7 Chapter Summary

Future research should therefore combine larger experimental datasets, more advanced Edge AI methods, enhanced FPGA processing, richer Digital Twin models, and scalable IIoT infrastructure.

These developments would move the current prototype toward a more complete and experimentally validated intelligent cable-harness monitoring platform.

Chapter 12

Conclusion

12.1 Chapter Overview

This project presented the design and implementation of a Real-Time Intelligent Cable Harness Test Bench integrating FPGA-based detection, Edge AI classification, Digital Twin state representation, and Industrial IoT monitoring.

The objective was to develop a modular architecture capable of combining deterministic embedded signal processing with higher-level intelligent classification and system monitoring.

12.2 Main Achievements

The FPGA subsystem was implemented on an AMD/Xilinx Artix-7 XC7A100T device. The final implementation achieved positive timing margin with a WNS of 0.093 ns and no failing timing endpoints, while maintaining low hardware resource utilization.

The Edge AI subsystem used a Random Forest classifier operating on 20 extracted features. The model achieved 94.5% classification accuracy and a macro F1-score of 0.938 on the defined scenario-level test set. The measured mean inference time was 0.014197 ms.

The IIoT layer integrated MQTT communication, FastAPI, InfluxDB, Grafana, and Docker Compose. The Digital Twin maintained the current system state and health score during the monitoring process.

The controlled end-to-end experiment processed 20 events, with four events from each defined class. Six FPGA detections were recorded, and the final Digital Twin health score reached 75.2%.

12.3 Overall Contribution

The main contribution of the project is the integration of multiple technologies into a single coherent cable-harness monitoring architecture.

The FPGA layer provides deterministic signal-level detection, the Edge AI layer provides event classification, the Digital Twin maintains a software representation of system condition, and the IIoT infrastructure provides communication, storage, and visualization.

This architecture demonstrates how embedded systems, machine learning, and cyber-physical monitoring can be combined to create a more intelligent approach to cable-harness testing.

12.4 Final Remarks

The developed system constitutes a functional prototype rather than a complete industrial product. Its current limitations, particularly the size and origin of the dataset and the scope of the Digital Twin, provide clear opportunities for further research.

Nevertheless, the project establishes a practical foundation for future development of intelligent cable-harness inspection systems combining FPGA processing, Edge AI, and connected monitoring infrastructure.

Bibliography

[1] ONNX Runtime, *ONNX Runtime Documentation*.

[2] OASIS, *MQTT Version 5.0*.

[3] FastAPI, *FastAPI Documentation*.

[4] Docker, *Docker Documentation*.

[5] Scikit-learn, *Machine Learning in Python*.

[6] Grafana Labs, *Grafana Documentation*.